

The background is a complex, abstract composition. It features a central grid of small, multi-colored squares (white, blue, yellow, purple, green) that transitions into a field of larger, 3D cubes. These cubes are scattered across the frame, some floating in the air and others forming a dense, stepped structure on the right side. The cubes are in various colors including white, grey, yellow, blue, green, and purple. The overall effect is a sense of digital architecture or data visualization.

Intent to <algorithm>

Turn design intentions into computational systems
using Grasshopper, rule-based thinking and AI-assisted coding.

About me



ADITYA SOMAN
B.Arch, MSc.
Design Application Developer

TU Delft
Delft, NL
MSc. Architecture , Building Sciences

Packhunt
Amsterdam , NL
Software Dev & Sorum Master

RAMBOLL
London , UK
Global Highrise & Complex Projects
Computational Design Specialist

TP BENNETT
London , UK
Compute
Software Dev

Design Orchestrator
An Agent-Based, AI-First Framework for Multidisciplinary Design Workflows

Aditya Soman¹, Jun Wen Luo²
¹Tp Bennett Architects, TBC_Engineers
²adityasoman@gmail.com, ¹lojunwen1995@gmail.com

Abstract. This paper presents an Agentic AI platform Design Orchestrator capable of building multi-disciplinary agentic workflows in a shared project context. The platform combines a node-based visual programming interface for constructing multi-capability agents with persistent project memory and integrations with computational design and analysis tools. This platform is evaluated through an early-stage design case study for a timber bridge limited to architectural and structural engineering disciplines. Two agents are developed on the platform one for site analysis and other for model exchange and structural analysis. The bridge design options are iteratively transferred, validated, structurally analysed, and evaluated for structural performance, embodied carbon, and relative cost. The case study demonstrates the benefits of integrating agentic design generation, analysis, evaluation, and documentation into a continuous workflow, enabling rapid design iterations while retaining human-in-the-loop validation at critical stages. It also reveals challenges related to agent reliability, requirement of precise agent instructions, performance optimization, context and token management, reinforcing the need for human oversight and more transparent agentic workflows.

Keywords. AI Agents, Multi-Disciplinary design, Agentic Collaboration, Data Context portability, Visual Programming.

INTRODUCTION
The rapid maturation of AI-first methodologies driven by advances in generative artificial intelligence and computational design is transforming contemporary architectural practice (Brynjolfsson et al., 2017). Emerging studies suggest that smaller, agile design teams adopting integrated AI workflows gain a competitive advantage as manual tasks become automated, data-rich, and interdependent. However, these advances often remain confined within disciplinary silos, limiting their impact on project delivery. Since the

Architecture, Engineering, and Construction (AEC) sector is inherently collaborative, the full benefits of AI integration can only be realised through collective, cross-disciplinary adoption rather than isolated innovation. Historically, architects have occupied the role of orchestrator coordinating diverse consultants, disciplinary expertise, and project knowledge to shape the overarching design intent. This paper introduces Design Orchestrator, an agent-based framework that extends this orchestration paradigm into a design ecosystem. Within this environment, all project disciplines

Machine Learning & Data-Driven Design - eCAADe 44 | 163

CONFERENCE PROGRAM

TOPOGEN
A generative design methodology for Building Engineering

Aditya Soman
Ramboll UK
aditya.soman@ramboll.co.uk

Topogen presents a novel methodology towards generative building engineering using the topological voxelization method to develop and analyze structural design options rapidly. In particular, the methodology showcases an automated, streamlined workflow for (1.) Extracting spatial data from input architectural models and embedding it in a voxelated version of the model (2.) Developing structural design options consisting of a 1D Frame and 2D shell elements generatively using a rule-based system to the discrete version of the model (3.) Developing accurate structural performance results for the input models by automatically performing a finite element analysis on the generated structural design options. The showcased approach opens up unexplored possibilities in generative design by reconsidering an existing concept for a different use case.

Co-creating the Future: Inclusion in and through Design

eCAADe 2022 Conference Proceedings
Sep. 13-16, 2022

edited by: Burak Pak, Gabriel Würzer and Rudi Stouffes

Vol. 1

DeciGenArch
A generative design methodology for participatory architectural configuration via multi-criteria decision analysis

Aditya Soman¹, Shervin Asadi², Pinot Nourou³
¹Ramboll UK, ²Tanderson University of Technology (TU) University of Technology
³aditya.soman@ramboll.co.uk, ²s.asadi@tucl.ac.uk, ³pinotnour@tudelft.nl

Our approach to Generative Design concerns the problems of design from the geometrical drawing of shapes in a continuous setting to topological decision making about spatial configurations in a discrete setting. The paper presents a comprehensive formulation of the zoning problem as a sub-problem of architectural 3D layout configurations. This formulation focuses on the problem of zoning as a location-allocation problem in the context of Operations Research. Specifically, we propose a methodology for solving this problem by combining a well-known Multi-Criteria Decision Analysis (MCDA) method called Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) with a Multi-Agent System (MAS) operating in a discrete design space.

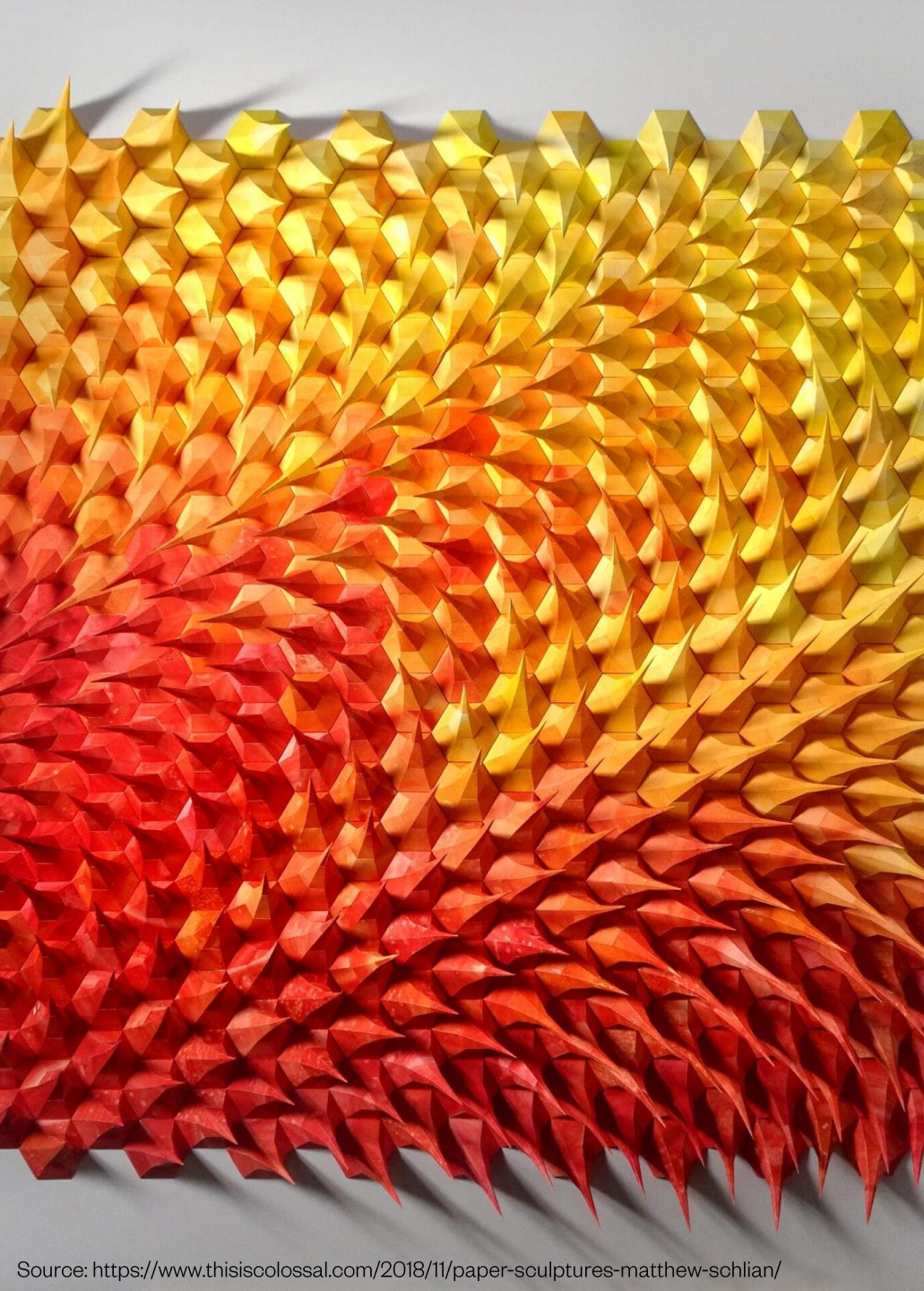
Keywords: 3D Layout problem, Participatory Design, Multi-Agent System, Multi-Criteria Decision Analysis, TOPSIS.

INTRODUCTION
The problem of architectural 3D layout configuration, when regarded in its full sophistication, could be regarded as a classical example of a 'wicked problem' (Rittel and Melrose, 1973). Leiber and Donath, 2010) due to the numerous human and physical complexities involved (Zawieska, 2016). Breaking this problem down into a set of subsequent problems through a methodical approach facilitates understanding and addressing the underlying complexities like multi-dimensional, multi-criteria, multi-actor, multi-value complexities holistically (Akad and Hourat, 2021). In this paper, the zoning problem, which is one of the most critical parts of the 3D layout problem, is studied in detail, and a methodology for resolving it is proposed. The zoning problem is equally one of the most essential parts of the 3D layout problem. Generating and locating the zones appropriately in the building mass has a massive impact on the building's functionality, efficiency, and utility for its occupants and their activities. Improper zoning can result in a poorly configured building, leading to many social, environmental, economic problems, which can be a shadow on the sustainability of the building in its life cycle.

The zoning problem in a continuous setting would be concerned with the shape of the spatial zones designated to the distinct activity spaces (also known as departments or facilities) as foreseen in a programme of requirements (collectively referred to as the so-called programmatic functions). In a discrete spatial setting, however, the design/decision space is already subdivided into volumetric units (voxels). This transforms the problem of zoning into a location-allocation problem, as it involves the allocation of and locating various zones in a discretized building mass corresponding to different programmatic functions. Simply put, such a location-allocation problem is

Volume 1 - Co-creating the Future - eCAADe 40 | 459

**“In the age of AI, the
real advantage is
knowing what’s
worth building.”**



CONTENTS

01. Introduction

- 1.1 AI state of the art today
- 1.2 LLM's
- 1.3 Agent's

02. Ideation

- 2.1 Basic's of Prompting
- 2.2 Prompt Engineering
- 2.3 Problem to Prompt

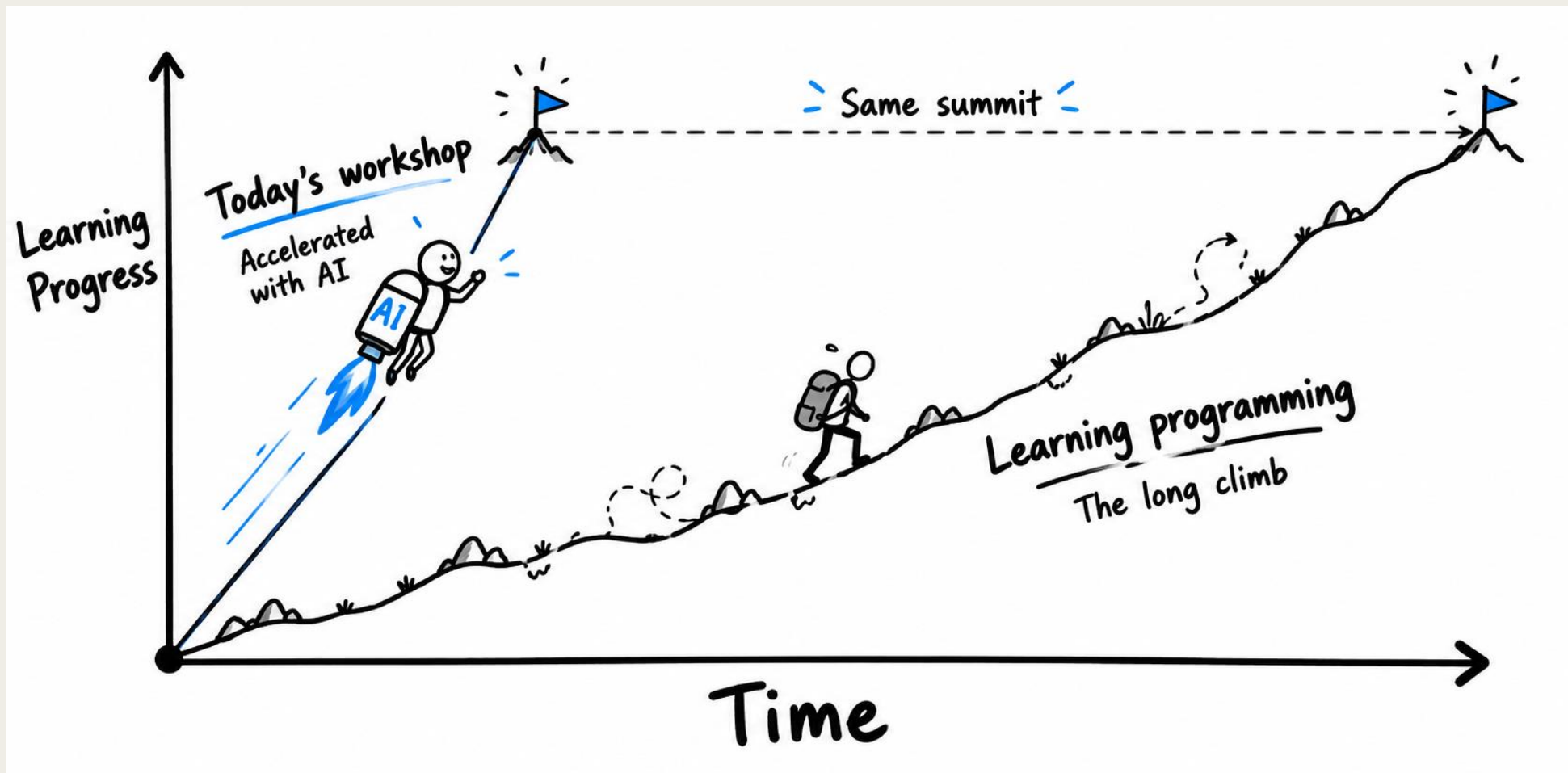
03. Scripting

- 3.1 Computer Programming
- 3.2 Python vs C#
- 3.3 Elements of code
- 3.4 Rhino Scripting Component
- 3.5 AI assisted Scripting

04. GIT and github

- 4.1 Workshop Demo

IN TODAY'S WORKSHOP



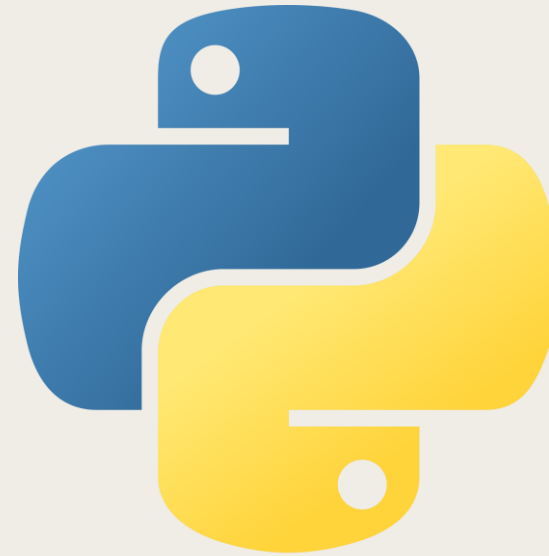
LEARNING PROGRAMMING IN 2 WEEKS IS VERY DIFFICULT... WE WILL TAKE AN ACCELERATED ROUTE

BUT MAKE SURE YOU LEARN THE BASICS AT YOUR OWN TIME !!

REQUIRED INSTALLATIONS



Rhino 8



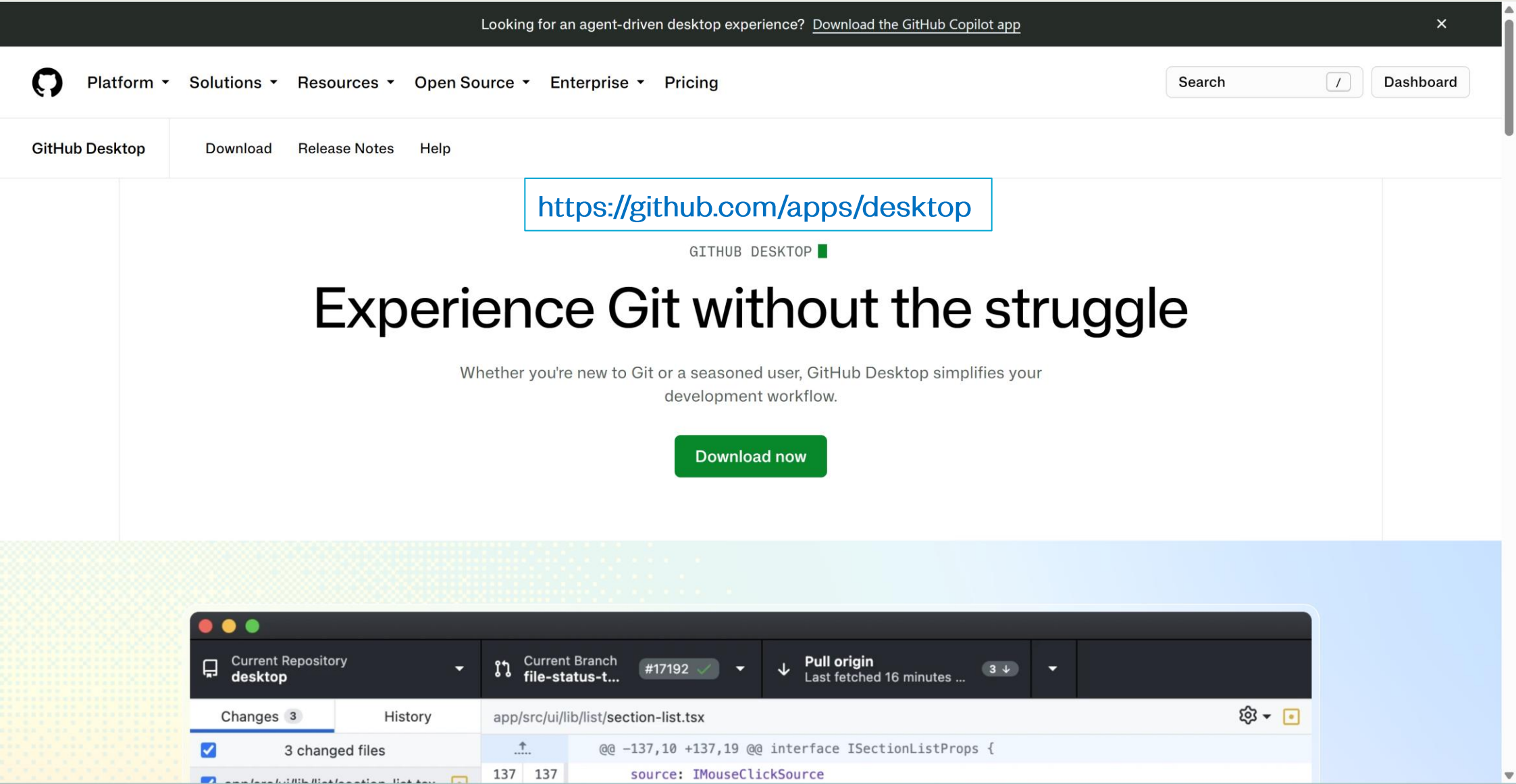
Python 3.14.8
* Installed with Rhino 8

REQUIRED INSTALLATIONS



* Create your github account and share details with me

REQUIRED INSTALLATIONS:



REQUIRED INSTALLATIONS:OPTIONAL

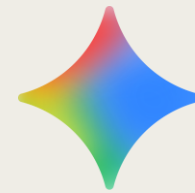
The image is a promotional banner for Visual Studio Code. At the top, there is a navigation bar with the Visual Studio Code logo and name, followed by links for Features, Docs, Release Notes, Blog, Learn, Events, and Resources. On the right side of the navigation bar, there is a search bar with the text "Search" and "Ctrl+Shift+P", and a "Download" button. Below the navigation bar, there is a light blue banner with the text "Rewatch GitHub Copilot Day." and a close button. The main background of the banner is a dark blue grid of binary code (0s and 1s). In the center, there is a large white text overlay that reads "The open source AI code editor" and "Your home for multi-agent development". Below this text, there is a dark blue button with the Visual Studio Code logo and the text "Download for Windows". Underneath the button, there is a link for "Web", "Insiders edition", or "other platforms". Below the link, there is a small text that says "By using VS Code, you agree to its license and privacy statement". At the bottom of the banner, there is a white box with the URL "https://code.visualstudio.com". In the bottom right corner, there is a screenshot of the Visual Studio Code editor interface. The screenshot shows a file named "MailList.tsx" with code that includes imports for "solid-js", "useMail", and "Item", and a constant "ROW" set to 56. To the right of the code editor, there is a panel with the title "REFACTOR MAILLIST COMPONENT" and a list of sessions. The first session is "Refactor MailList component" with the status "Thinking...". The second session is "Wire MCP server for telemet..." with the status "Editing mcp.config.ts". The third session is "Add language server for .pri..." with the status "Running tests in parser.test.ts".

REQUIRED INSTALLATIONS:



ChatGPT

OR



Gemini

OR

* Any AI agent with image
generation functionality

OUR WORKSHOP REPOSITORY

adityasoman / UCL26Workshops-Intent-To-Algorithm

Type / to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

UCL26Workshops-Intent-To-Algorithm

Public

Pin

Watch 0

Fork 0

Star 0

main 1 Branch 0 Tags

Go to file

Add file

<> Code

adityasoman moved directory of folders d8aefe8 · now 4 Commits

Group-Assignments	moved directory of folders	now
Individual-Assignments	moved directory of folders	now
Intent-to-Algorithm	moved directory of folders	now

https://github.com/adityasoman/UCL26Workshops-Intent-To-Algorithm.git

README

Intent to <algorithm>

Turn design intentions into computational systems using Grasshopper, rule-based thinking and AI-assisted coding. Python module, The Bartlett School of Architecture

🚧 Facilitator guide in progress. Lesson order, timings and talking points will be added here once all the lessons are written.

About

This is a repository which hosts all the content shared by the Tutors and Students both for Intent-To-Algorithm workshop

Readme

Activity

0 stars

Releases

No releases published

[Create a new release](#)

Packages

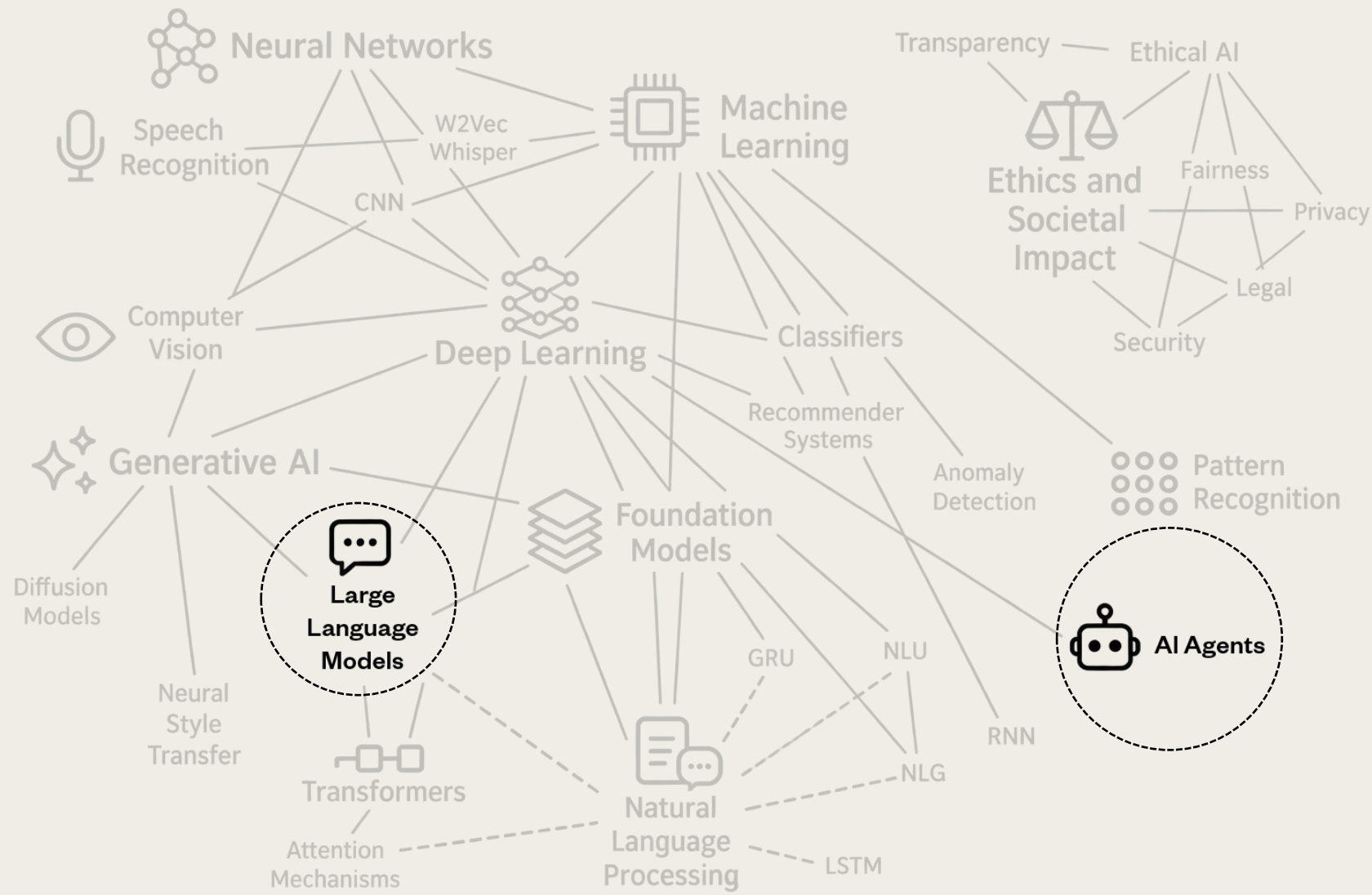
No packages published

[Publish your first package](#)

Contributors 2

adityasoman Aditya Soman

1.1 AI STATE OF THE ART





What is an LLM?

1.2 LLM'S

Text is split into tokens:

These are words or parts of words. The LLM converts each token into a numerical representation called an embedding.

Most LLMs use transformers:

Their attention mechanism helps connect relevant tokens for example, identifying what "it" refers to in a sentence.

Training adjusts billions of parameters:

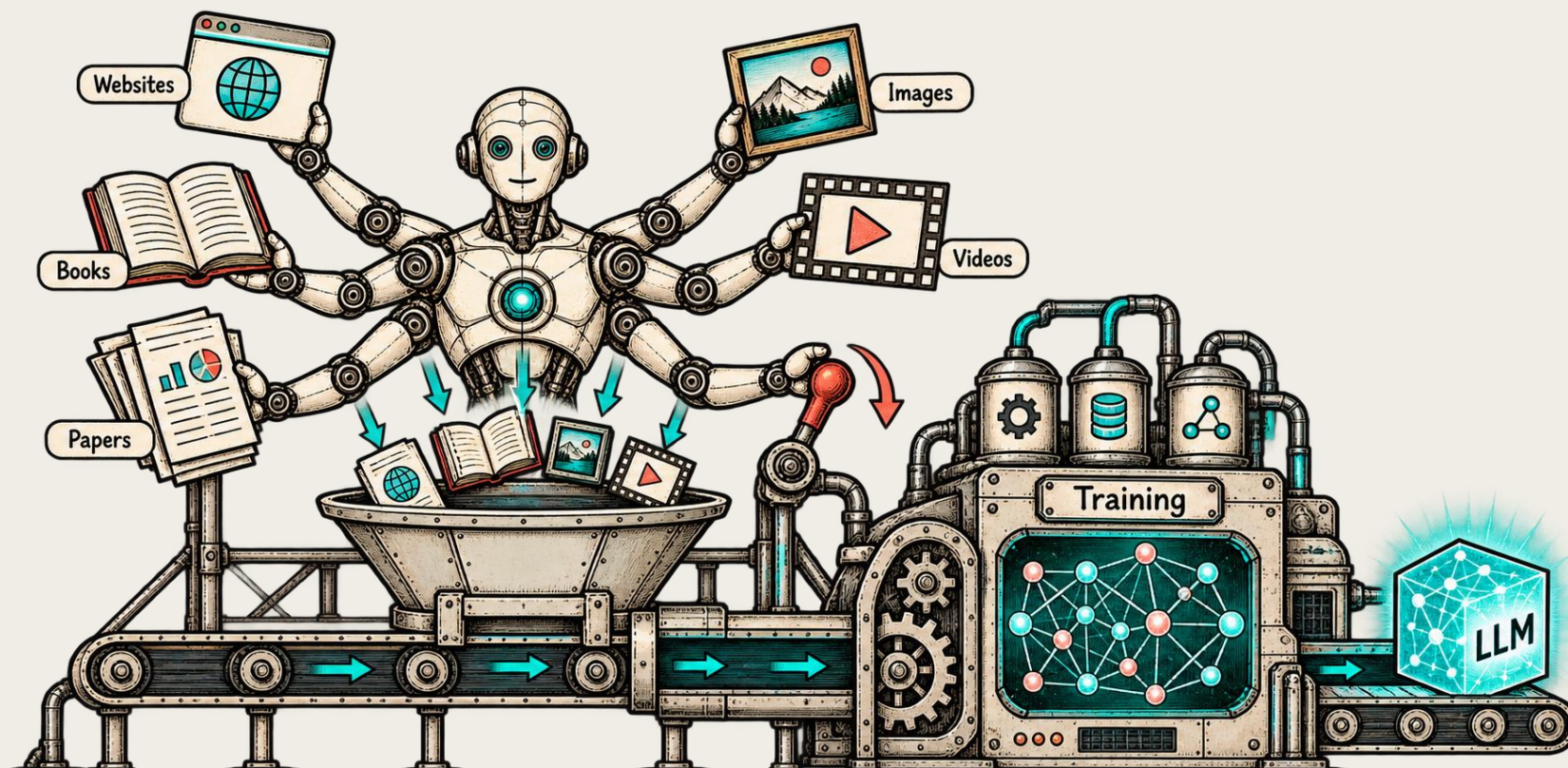
These internal numerical values are tuned to improve predictions, commonly by predicting the next token across large amounts of text.

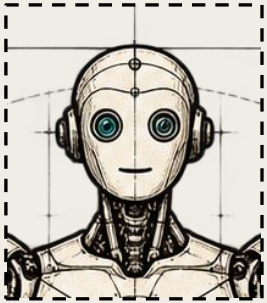
Answers are generated token by token:

The model assigns probabilities to possible next tokens and selects one repeatedly. Its context window limits how much input and previous conversation it can process at once.

LLM = Large Language Model

an AI model that processes and generates human language.

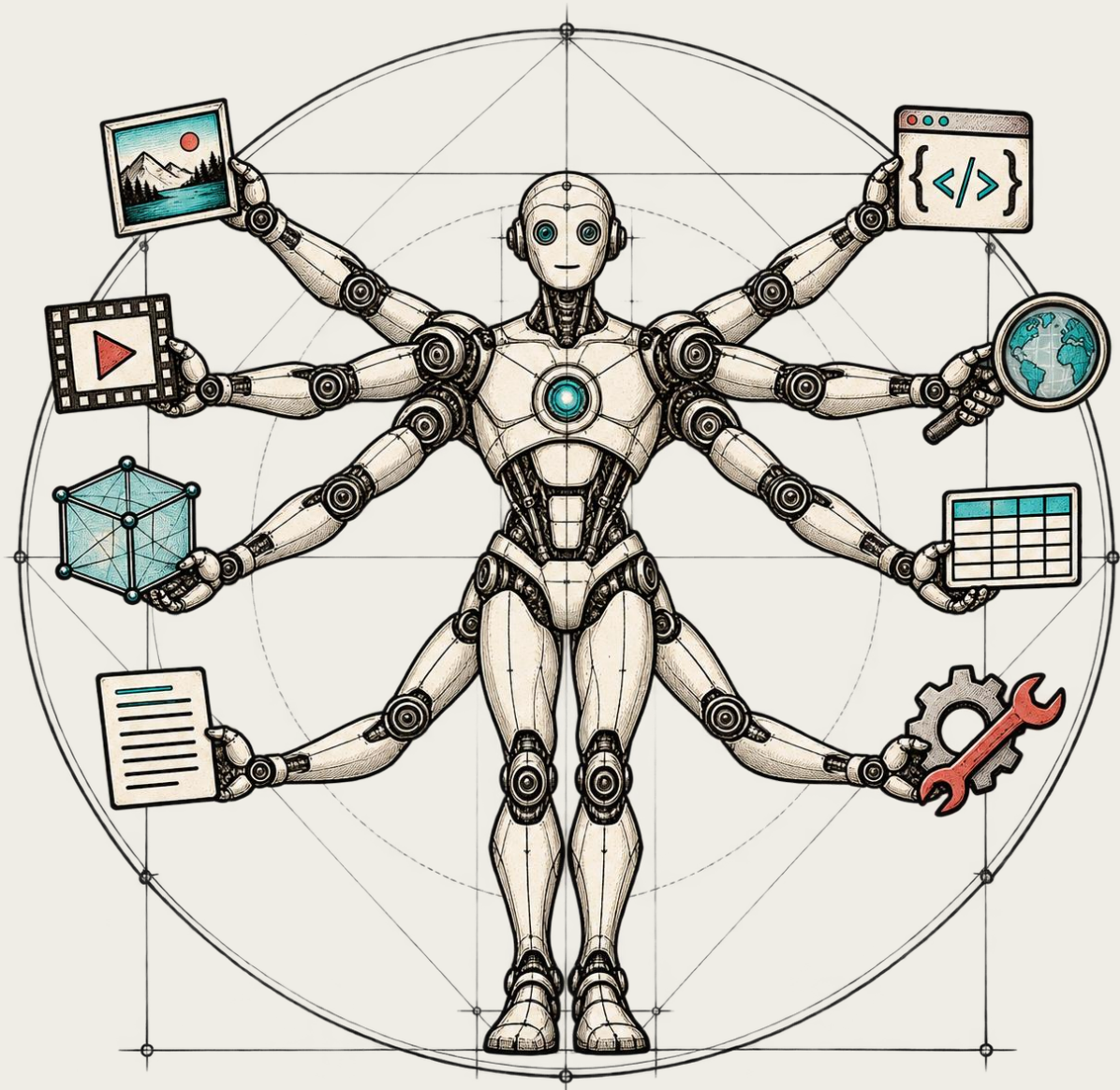




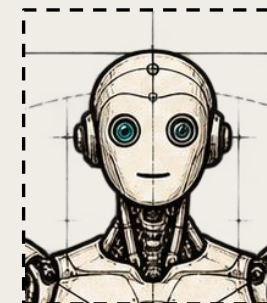
What is an Agent?

**‘Agent is anything that can
be considered able to
perceive its environment
and take actions’**

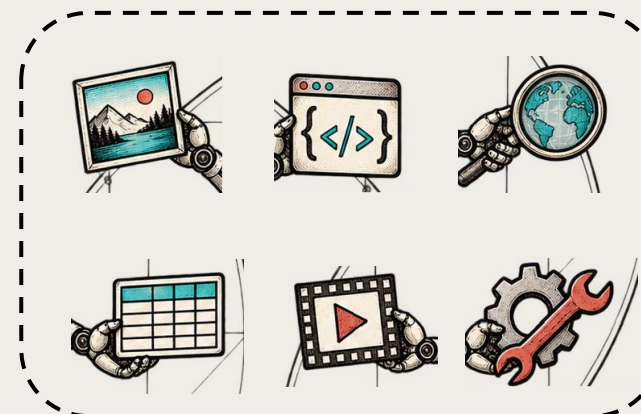
AGENTS



LLM is the Brain



?? Reason and Call Tools



AGENTS

AGENT

What should we work on?

Work with ChatGPT

LLM

+

GPT-6.1 Sol Light ▾



Choose project

Files

Plugins

Open desktop app

CHATGPT is Interface



Based on Your prompt

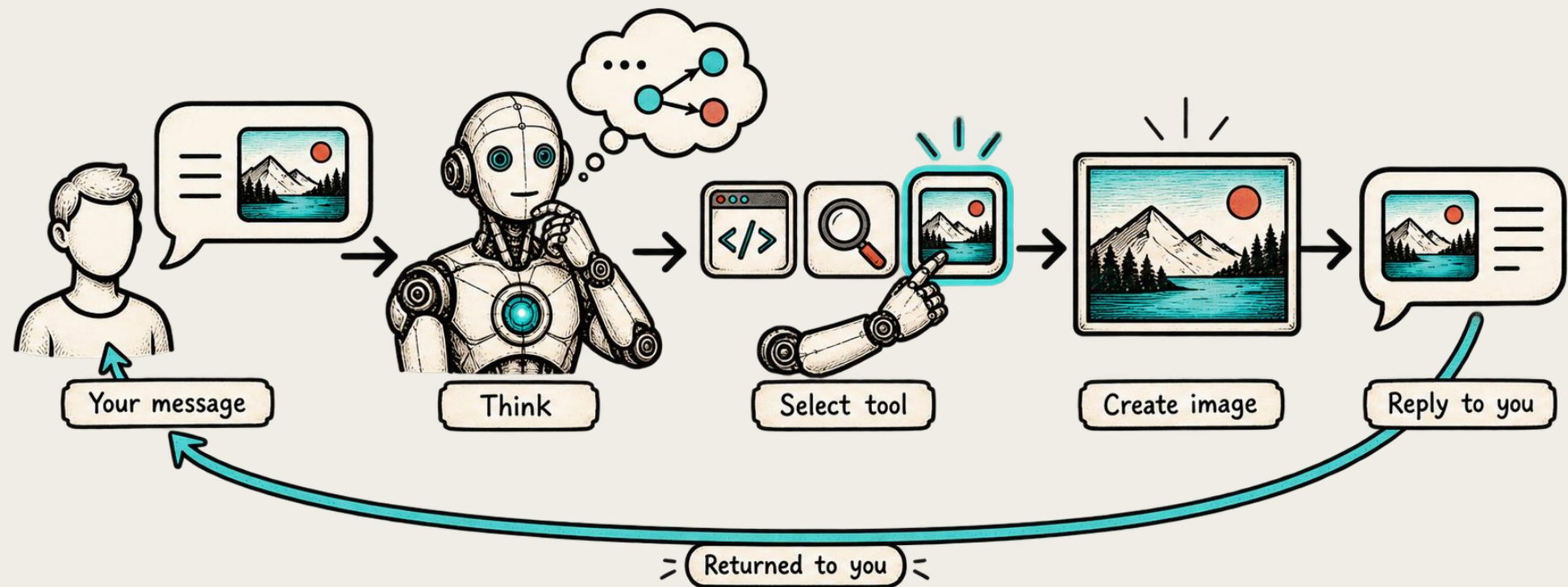


Selects an LLM



Calls an Agent

AGENTS



CODING AGENTS

Popular coding agents from frontier labs



CLAUDE CODE



OPENAI CODEX

Others Include
Github Co-Pilot
Cursor
Junie
OpenCode
AntiGravity

...

RECAP

AFTER A SHORT COFFEE BREAK...

02. IDEATION

2.1 BASICS OF PROMPTING

Define the objective :

Use a precise instruction such as “summarise,” “classify,” or “debug,” and describe what success looks like

Most LLMs use transformers:

Include inputs, background, and assumptions. Separate instructions from source material using labels

Define constraints and outputs:

Define scope, length, tone, and structure

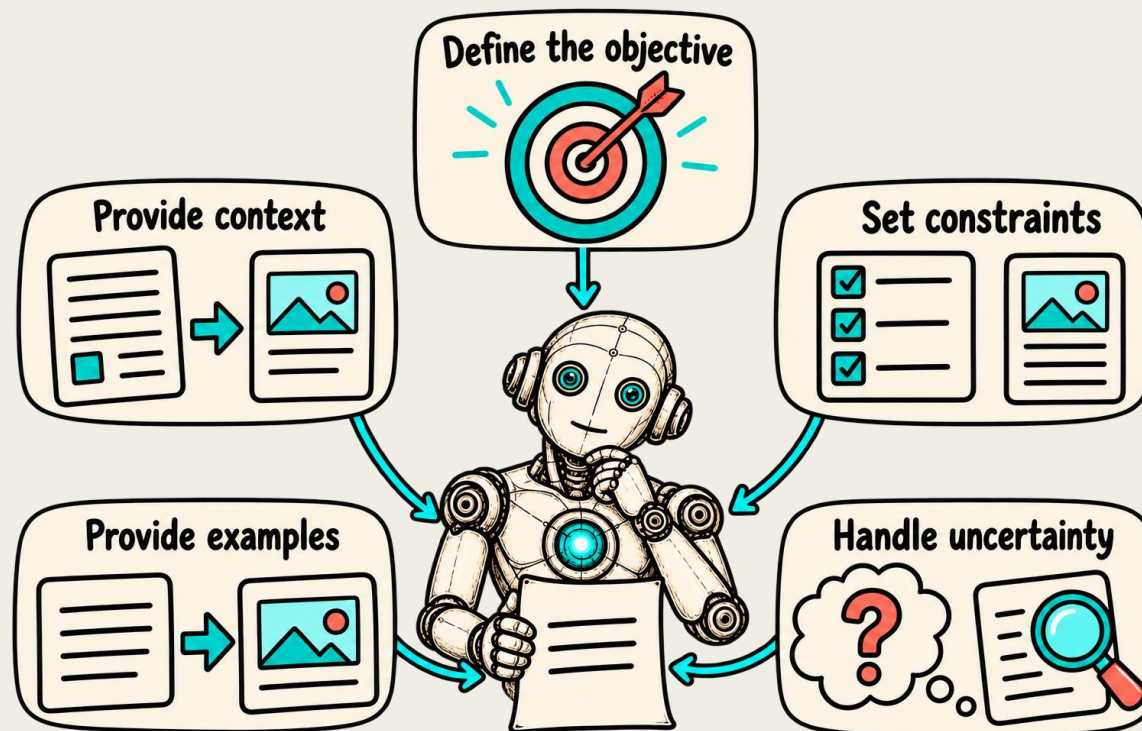
Provide examples:

Show one or two sample inputs and desired outputs. This is called few-shot prompting.

Handle Uncertainty:

Ask the model to identify missing information, state assumptions, and avoid inventing facts.

Be as Descriptive and clear as you can



2.2 PROMPT ENGINEERING

Purpose:

Prompt engineering is the systematic design, testing, and refinement of instructions and context to guide an AI model toward a desired output.

Relevance Today:

Even capable models need clear goals and relevant information, especially for complex tasks and specialised work.

For Agents:

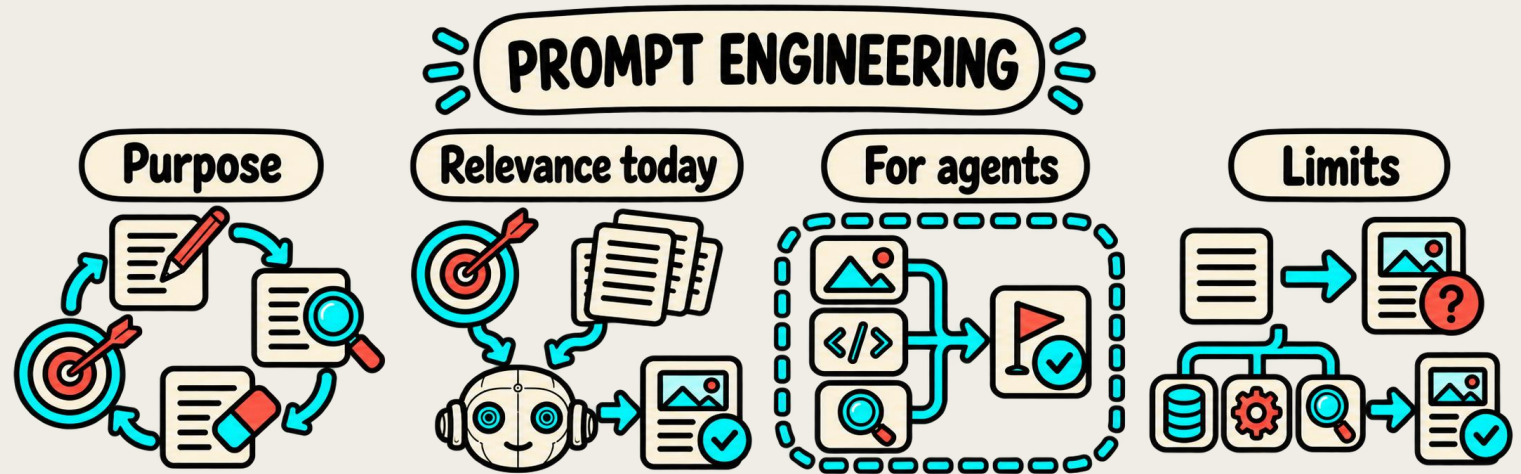
It helps define tool use, boundaries, and completion criteria when AI carries out tasks.

Limit:

Good prompts help, but they cannot guarantee accuracy; reliable systems also need suitable data, tools, and validation.

Prompt engineering

Systematic design, testing, and refinement of instructions and context to guide an AI model toward a desired output



2.3 PROBLEM TO PROMPT

EXAMPLE : RENDER A WHITECARD

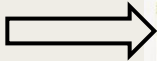


2.3 PROBLEM TO PROMPT

MODELS ARE VERY GOOD AT REASONING TODAY

PROMPT: CREATE A PHOTOREALISTIC RENDERING

Windows and daylight are assumed



Undefined materials or objects like the **existing Cast Iron columns** that were not detected were kept white-card

Materials assumed

Model made an assumption

Created a photorealistic rendering with natural oak furniture, neutral finishes, and warm daylight, retaining the original architectural layout.

2.3 PROBLEM TO PROMPT

Task:

Transform the attached whitecard image into a photorealistic rendering of a contemporary office interior for a client design presentation.

Reference:

Use the image as the geometric reference. Preserve the communal tables and chairs, rear workstations, left-side corridor and glazed partition, columns, exposed ceiling services, and suspended lighting.

Material specifications:

Communal tabletops: natural pale oak veneer with fine, visible grain and a matte clear finish. Preserve existing tabletop shapes and thicknesses.

Table legs and frames: warm-white powder-coated steel with a fine-textured matte finish.

Foreground chairs: pale oak frames with muted sage woven-fabric seat pads. Preserve each chair’s original form; use satin brushed metal for visible tubular frames.

Rear task chairs: charcoal woven upholstery with matte graphite bases and armrests.

Workstation surfaces and storage: warm-white matte laminate with pale oak accents; avoid glossy plastic.

Workstation screens: muted sage acoustic felt with a soft, fine-fibre texture.

Communal and corridor flooring: light grey concrete-look porcelain tiles, approximately 600 × 600 mm, with narrow matching grout joints and a matte finish.

Workstation flooring: medium-grey low-pile carpet tiles, approximately 500 × 500 mm, with subtle tonal variation and flush transitions to tiled areas.

Walls and columns: warm-white matte paint.

Glazed partition: clear glass.

Ceiling, beams, and ductwork: warm-white matte paint.

Exposed mechanical services: black-painted steel.

Suspended lighting: warm-white LED.

Styling:

Create a calm, professional atmosphere.

APPROACH 1: DETAILED PROMPT COVERING ALL NECESSARY DETAILS
IN THE FIRST PROMPT

Lighting and realism:

Use soft daylight consistent with the visible openings, balanced with illumination from the existing fixtures. Render physically plausible material roughness, reflections, contact shadows, and subtle imperfections. Keep wood grain, fabric weave, tile joints, and carpet texture at realistic scales. Avoid repeating texture patterns, excessive gloss, oversaturation, and a stylised CGI appearance.

Constraints:

Maintain the original camera position, perspective, framing, and 16:9 aspect ratio. Preserve room proportions, furniture placement, chair designs, columns, partitions, circulation routes, and ceiling equipment. Do not add windows, doors, structural elements, or people. Keep vertical lines straight.

Assumptions:

These materials are proposed design finishes, not verified existing specifications. Where an object is unclear, apply a simple finish consistent with this palette without changing its geometry.

Output and success criteria:

Produce one high-resolution photorealistic image suitable for presentation. The result must match the whitecard’s composition and layout, with convincing material scale, natural lighting, and clearly distinguishable finishes.

2.3 PROBLEM TO PROMPT

Detailed Structured
prompt



Lower Chances of
Hallucinations &
Assumptions



Faster Desired
Outputs

PROMPT: DETAILED PROMPT FROM LAST SLIDE



You get a lot of things correct on the first attempt. Real materials, daylighting conditions and image outputs

2.3 PROBLEM TO PROMPT

EXAMPLE : RENDER A WHITECARD



/ ASK CLARIFYING QUESTIONS

Yesterday 7:15 PM



Render this whitecard image and ask clarifying questions one at a time. Do not make any assumptions

Worked for 16s >

What visual style would you like for the finished render?



photorealistic

What materials and colours would you like for the floors, walls, desks, and storage units?



Floors as Vitrified tiles walls exposed brick desk warm oak and storage units in metal

What material and colour would you like for the desk divider panels?



make assumptions for rest

Worked for 1m 16s >



Edit



Rendered with your chosen materials, warm white lighting, and muted green fabric dividers.



2.3 PROBLEM TO PROMPT

EXERCISE 01: USE AN AI AGENT OF YOUR CHOICE FOR THE FOLLOWING ASSIGNMENT

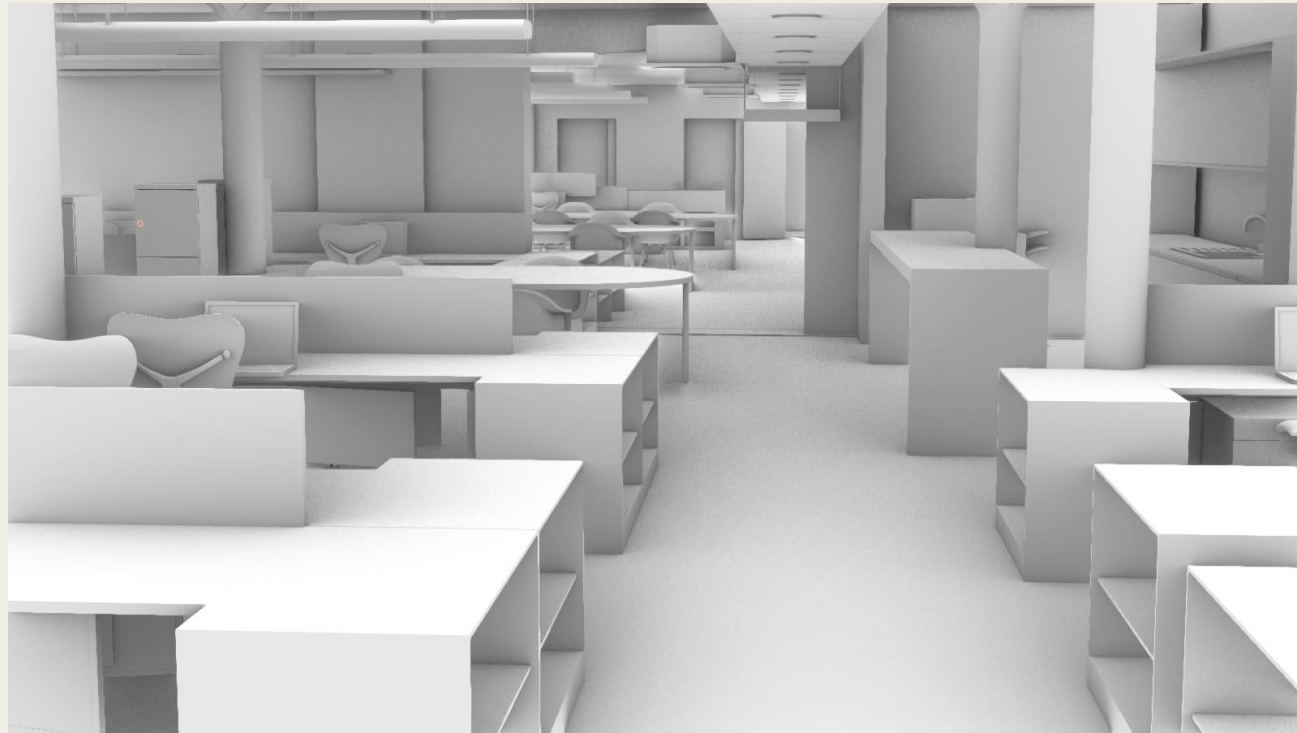
*Recommended Agent should have access to image generation tools.

Chatgpt or Gemini both have image generation tools

2.3 PROBLEM TO PROMPT

APPROACH 2: / ASK CLARIFYING QUESTIONS

PROMPT : Render this white card image and ask clarifying questions one at a time. Do not make any assumptions



Intent-to-Algorithm/Ideation-Exercises/Exercise-01

2.3 PROBLEM TO PROMPT

Github: Intent-to-Algorithm/Ideation-Exercises/Exercise-02/Ask_Questions.md

I am an architecture student learning {Add your own text}.
My design intention is: {describe your idea in one or two sentences}.
{ either add some of the solutions you are looking at or ask AI to suggest }
Before suggesting any solution, ask me clarifying questions
ONE AT A TIME. Wait for my answer before asking the next one.
Do not make any assumptions.

EXERCISE 02: USING ASK QUESTION TO CLARIFY / IDEATE

1. Pick an intention from a project or Question of your choice.
2. Use **the basic prompt** above and answer at least 5 questions.
3. Type `SUMMARISE`.
4. Compare with others : did the AI ask you the same questions?
Which question changed your design the most?

RECAP

AFTER A SHORT COFFEE BREAK...

03. SCRIPTING FUNDAMENTALS

3.2 PYTHON VS C#

Simple syntax

Used for AI, data analysis, automation, and web development.

Dynamic typing: flexible, but some errors appear when running.

Quick to write and many useful libraries. Huge Community. AI is essentially built on PY

Slower to compute dependencies can be tricky to manage.



*Supported by python foundation

VS



*Supported by Microsoft

More rules and syntax to learn.

Used for desktop and windows applications.

Static typing: catches many errors before running.

Runs faster, strong tooling, and good for large applications.

More setup and code for simple tasks.

3.1 COMPUTER PROGRAMMING

Our aim:

Learning computer programming in two weeks is a mammoth task.

We will focus on understanding key Python fundamentals.

01. Data structures:

Organise and store information using lists, tuples, dictionaries, and sets.

02. Imports and modules:

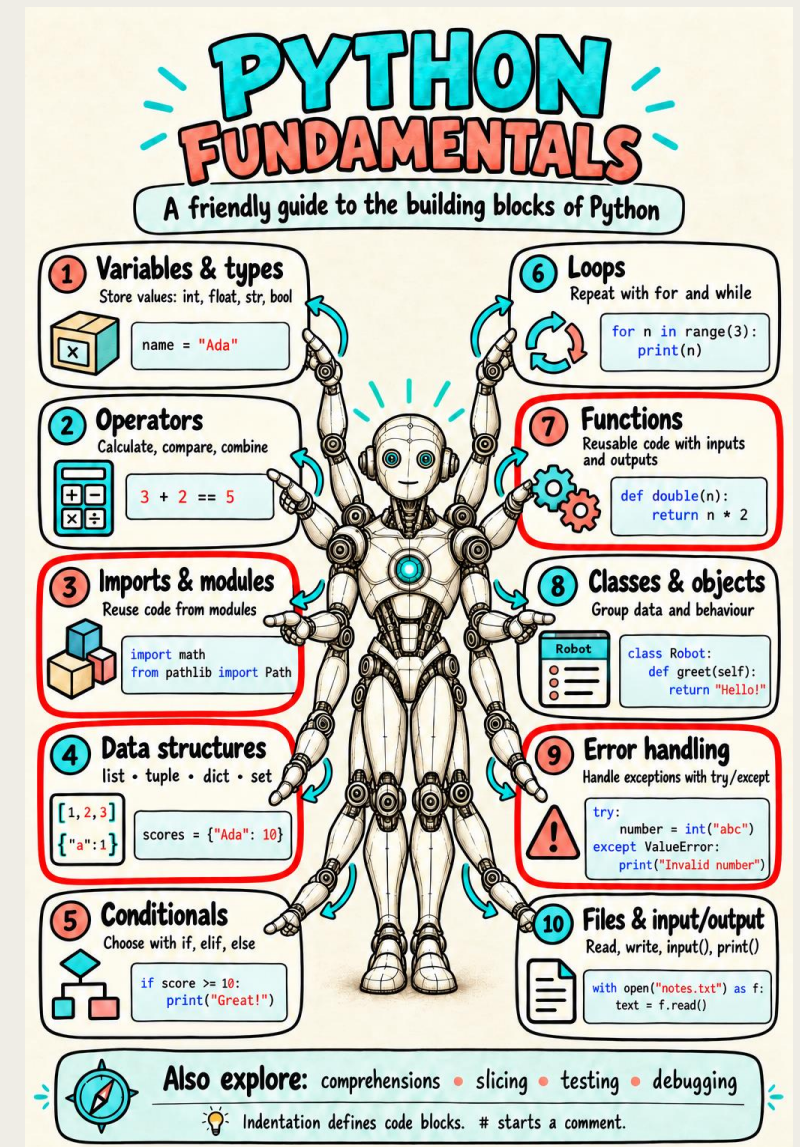
Access existing code and libraries to use in our programs.

03. Functions:

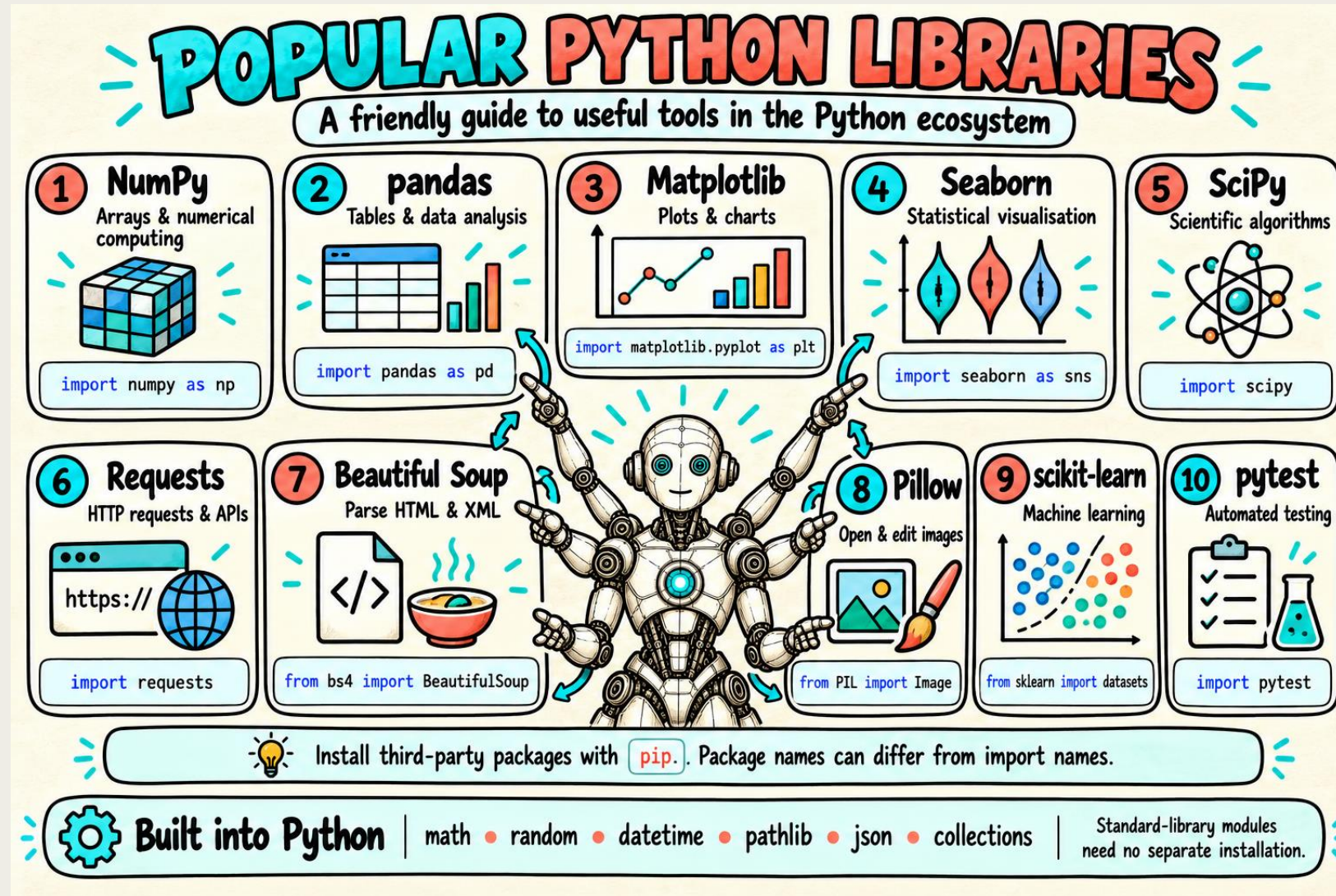
Group code into reusable blocks that perform specific tasks.

04. Error handling:

Recognise and manage errors so our programs can respond appropriately.



3.2 PYTHON POPULAR LIBRARIES



3.2 PYTHON CHEAT SHEET



Real Python Pocket Reference

Visit realpython.com to turbocharge your Python learning with in-depth tutorials, real-world examples, and expert guidance.

Getting Started

Follow these guides to kickstart your Python journey:

- realpython.com/what-can-i-do-with-python
- realpython.com/installing-python
- realpython.com/python-first-steps

Start the Interactive Shell

```
$ python
```

Quit the Interactive Shell

```
>>> exit()
```

Run a Script

```
$ python my_script.py
```

Run a Script in Interactive Mode

```
$ python -i my_script.py
```

Learn More on realpython.com/search:

[interpreter](#) · [run a script](#) · [command line](#)

Comments

- Always add a space after the #
- Use comments to explain "why" of your code

Write Comments

```
# This is a comment
# print("This code will not run.")
print("This will run.") # Comments are ignored by Python
```

Learn More on realpython.com/search:

[comment](#) · [documentation](#)

Data Types

- Python is dynamically typed
- Use `None` to represent missing or optional values
- Use `type()` to check object type
- Check for a specific type with `isinstance()`
- `issubclass()` checks if a class is a subclass

Type Investigation

```
type(42)           # <class 'int'>
type(3.14)         # <class 'float'>
type('Hello')     # <class 'str'>
type(True)        # <class 'bool'>
type(None)        # <class 'NoneType'>

isinstance(3.14, float) # True
issubclass(int, object) # True - everything inherits from object
```

Type Conversion

```
int("42")          # 42
float("3.14")      # 3.14
str(42)            # "42"
```

Variables & Assignment

- Variables are created when first assigned
- Use descriptive variable names
- Follow `snake_case` convention

Basic Assignment

```
name = "Leo"       # String
age = 7            # Integer
height = 5.6       # Float
is_cat = True      # Boolean
flaws = None       # None type
```

Parallel & Chained Assignments

```
x, y = 10, 20      # Assign multiple values
a = b = c = 0      # Give same value to multiple variables
```

Augmented Assignments

```
counter += 1
numbers += [4, 5]
permissions |= write
```

Learn More on realpython.com/search:

[variables](#) · [assignment operator](#) · [walrus operator](#)

Strings

- It's recommended to use double-quotes for strings
- Use `"\n"` to create a line break in a string
- To write a backslash in a normal string, write `"\\"`

Creating Strings

```
single = 'Hello'
double = "World"
multi = """Multiple
line string"""
```

String Operations

```
greeting = "me" + "ow!" # "meow!"
repeat = "Meow!" * 3    # "Meow!Meow!Meow!"
length = len("Python") # 6
```

String Methods

```
"a".upper() # "A"
"A".lower() # "a"
"a".strip() # "a"
```

String Indexing & Slicing

```
text = "Python"
text[0] # "P" (first)
text[-1] # "n" (last)
text[1:4] # "yth" (slice)
text[:3] # "Pyt" (from start)
text[3:] # "hon" (to end)
text[::-2] # "Pto" (every 2nd)
text[::-1] # "nohtyP" (reverse)
```

String Formatting

```
# f-strings
name = "Aubrey"
age = 2
f"Hello, {name}!" # "Hello, Aubrey!"
f"{name} is {age} years old" # "Aubrey is 2 years old"
f"Debug: {age}" # "Debug: age=2"

# Format method
template = "Hello, {name}! You're {age}."
template.format(name="Aubrey", age=2) # "Hello, Aubrey! You're 2."
```

Raw Strings

```
# Normal string with an escaped tab
"This is:\tCool." # "This is: Cool."

# Raw string with escape sequences
r"This is:\tCool." # "This is:\tCool."
```

Learn More on realpython.com/search:

[strings](#) · [string methods](#) · [slice notation](#) · [raw strings](#)

Numbers & Math

Arithmetic Operators

```
10 + 3 # 13
10 - 3 # 7
10 * 3 # 30
10 / 3 # 3.3333333333333335
10 // 3 # 3
10 % 3 # 1
2 ** 3 # 8
```

Useful Functions

```
abs(-5) # 5
round(3.7) # 4
round(3.14159, 2) # 3.14
min(3, 1, 2) # 1
max(3, 1, 2) # 3
sum([1, 2, 3]) # 6
```

Learn More on realpython.com/search:

[math](#) · [operators](#) · [built in functions](#)

Conditionals

for code blocks

Use 4 spaces per indentation level

If-Elif-Else

```
if age < 13:
    category = "child"
elif age < 20:
    category = "teenager"
else:
    category = "adult"
```

Comparison Operators

```
x == y # Equal to
x != y # Not equal to
x < y # Less than
x <= y # Less than or equal
x > y # Greater than
x >= y # Greater than or equal
```

Logical Operators

```
if age >= 18 and has_car:
    print("Roadtrip!")

if is_weekend or is_holiday:
    print("No work today.")

if not is_raining:
    print("You can go outside.")
```

Learn More on realpython.com/search:

[conditional statements](#) · [operators](#) · [truthy falsy](#)

Cheat sheets are good ways to learn new Libraries and Languages

🌱 Continue your learning journey and become a Python expert at realpython.com/start-here



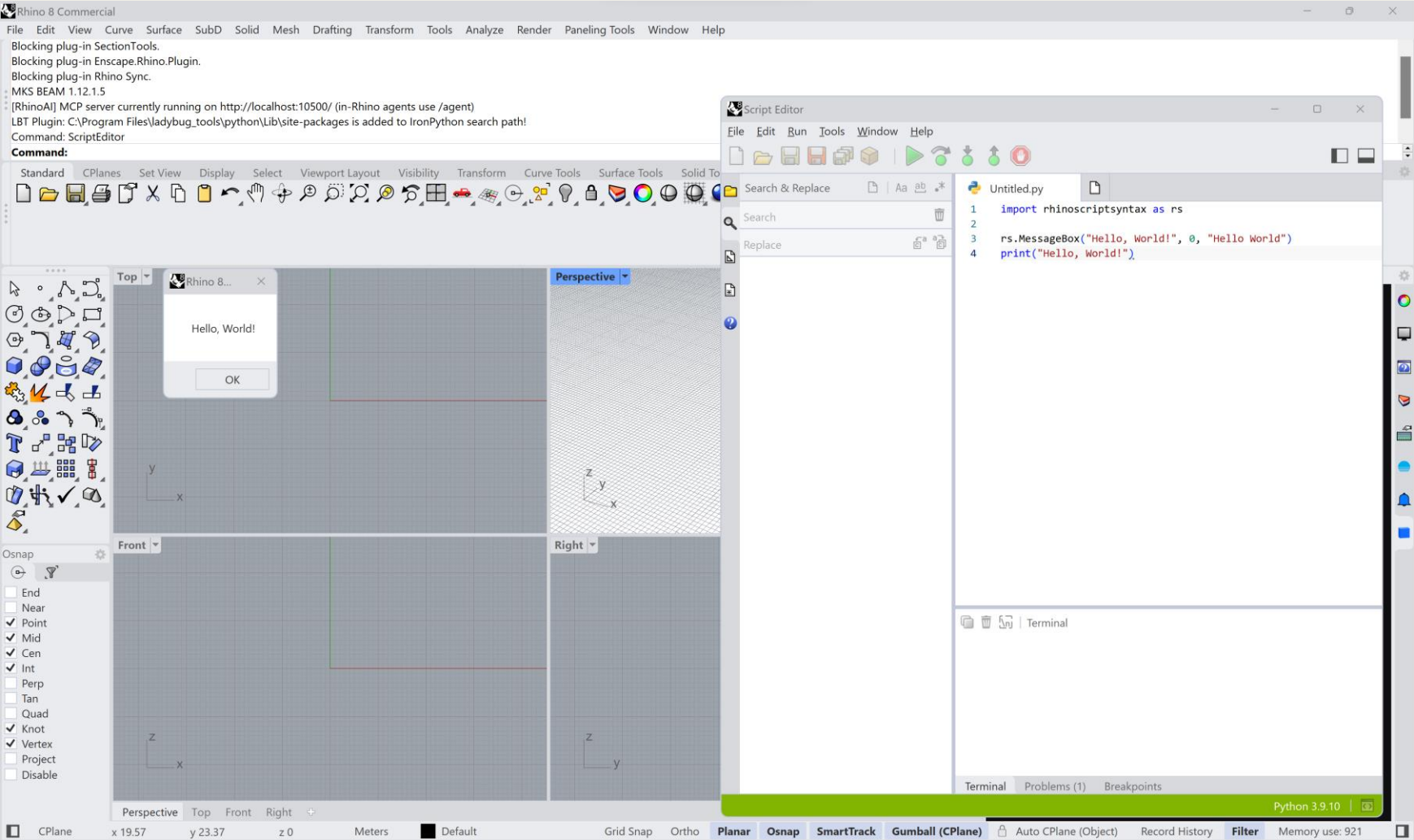
RECAP

AFTER A SHORT COFFEE BREAK...

HOW DO WE SCRIPT IN RHINO ?

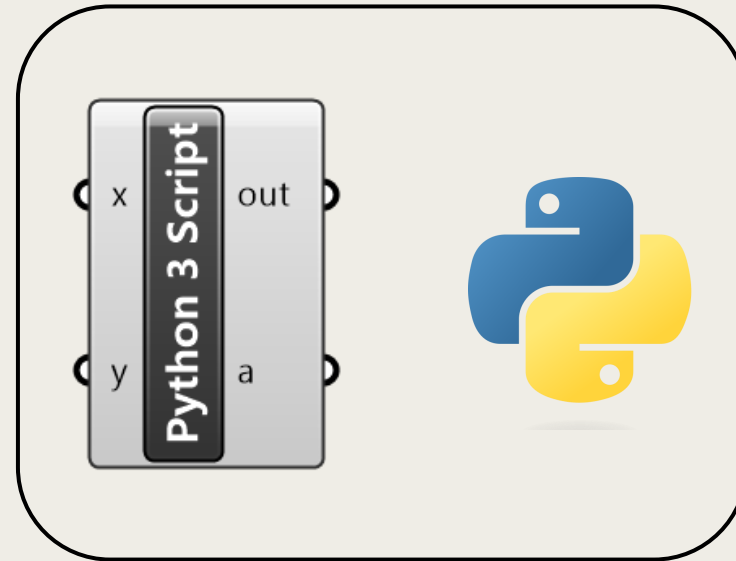
3.3 RHINO SCRIPT COMPONENT

Rhino Command: ScriptEditor



Live Demo.

3.3 GRASSHOPPER SCRIPT COMPONENT



Live Demo: Open Grasshopper

3.3 TWO WAYS TO FOLLOW

RhinoScriptEditor



Intent-to-Algorithm/RhinoScript editor

UCL26Workshops-Intent-To-Algorithm / Intent-to-Algorithm / RhinoScript editor /			Add file ▾	⋮
adityasoman added new technique			b09cac2 · 44 minutes ago History	
Name	Last commit message	Last commit date		
..				
00_hello_world	modified md	yesterday		
01_data_types	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
02_imports	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
03_console_errors	added new technique	44 minutes ago		
04_functions	Added new Task	20 hours ago		
05_conditionals	modified md	yesterday		
06_loops	modified md	yesterday		
07_classes	modified md	yesterday		
08_data_input_output	Added art installation example	yesterday		
09_capstone_producer_operator_consumer	Added art installation example	yesterday		
_TEMPLATE	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
README.md	Added art installation example	yesterday		

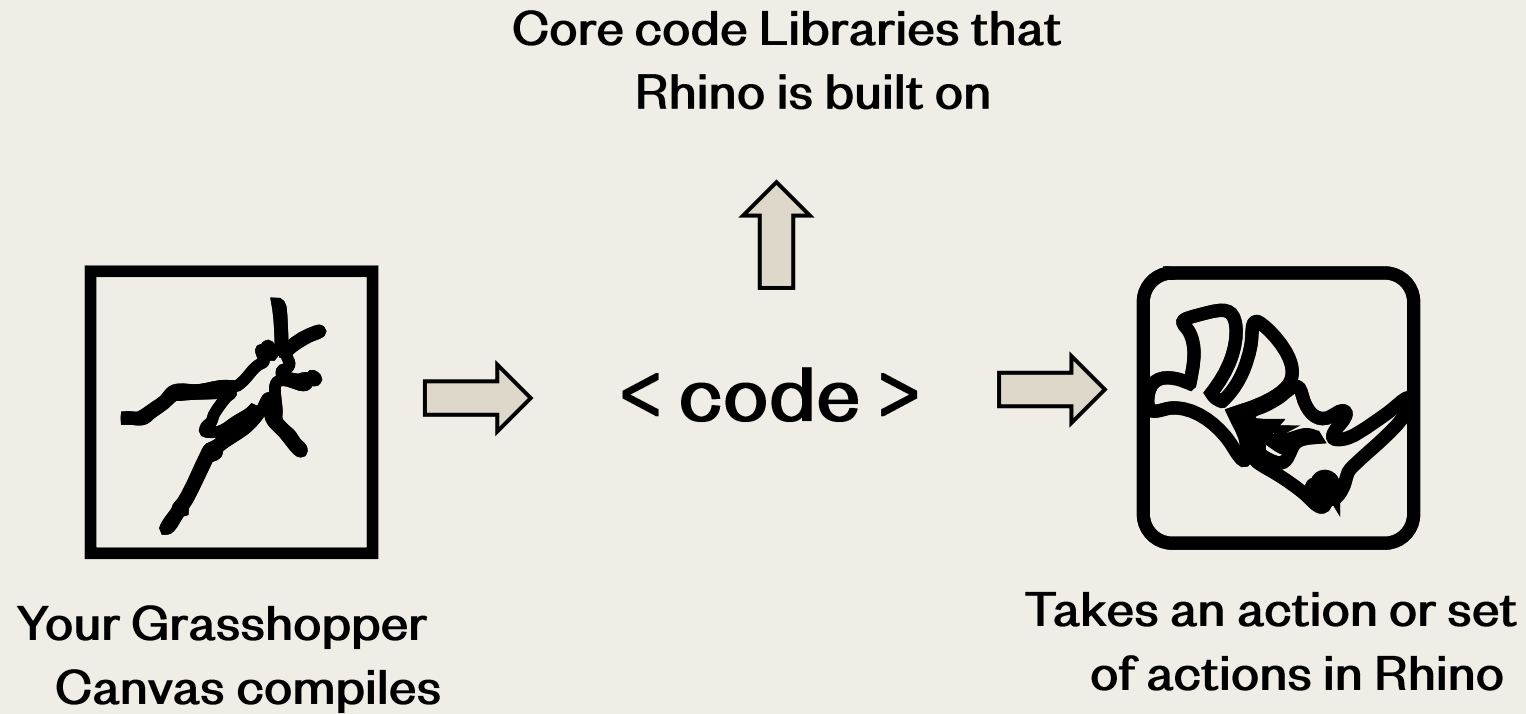
Grasshopper Script



Intent-to-Algorithm/Grasshopper scripts

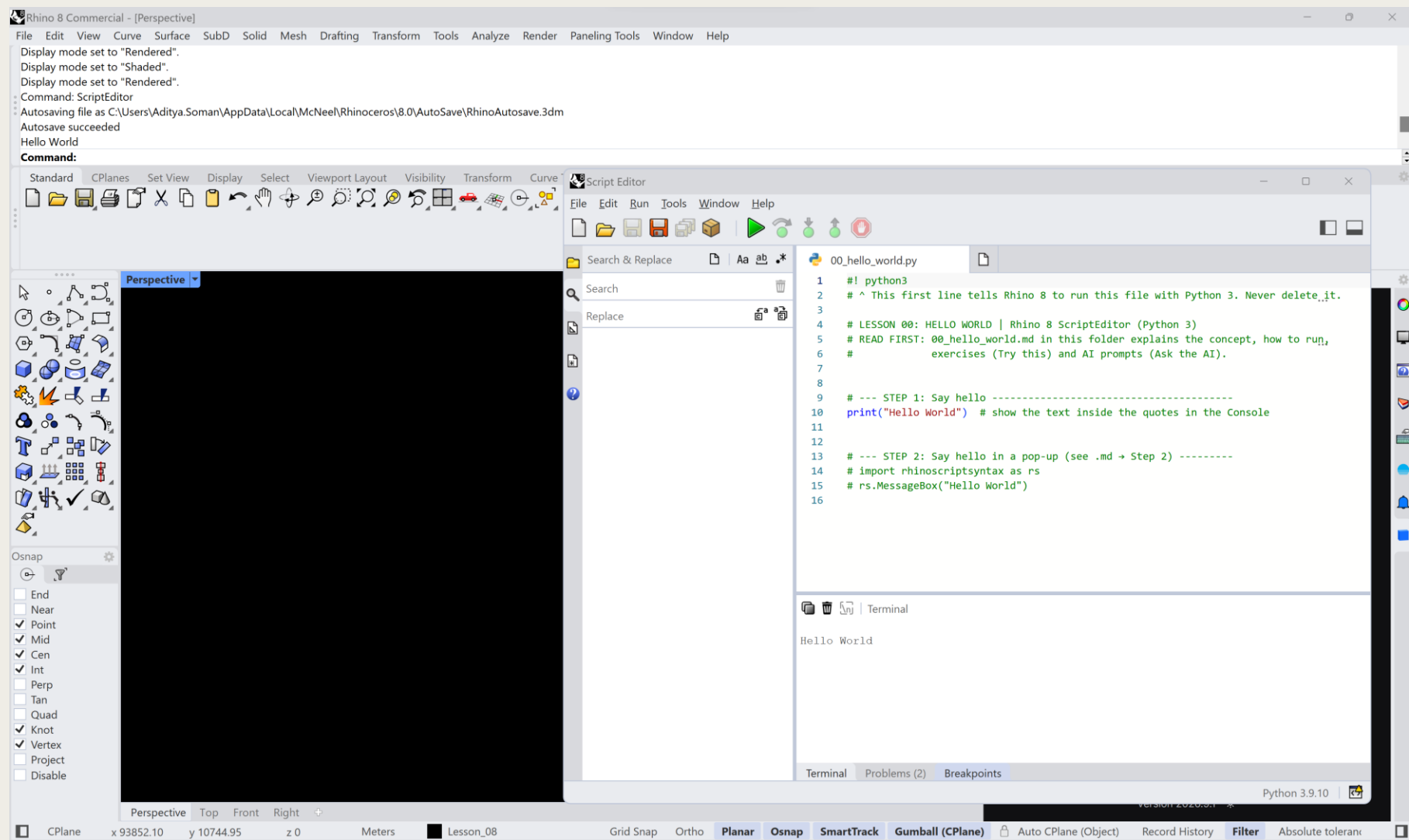
UCL26Workshops-Intent-To-Algorithm / Intent-to-Algorithm / Grasshopper scripts /			Add file ▾	⋮
adityasoman Added some more markdowns to tutorials			92e2c8c · yesterday History	
Name	Last commit message	Last commit date		
..				
00_hello_world	Added more examples	yesterday		
01_data_types	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
02_imports	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
03_console_errors	Added more examples	yesterday		
04_functions	Added some more markdowns to tutorials	yesterday		
05_conditionals	modified md	yesterday		
06_loops	modified md	yesterday		
07_classes	modified md	yesterday		
08_data_input_output	modified md	yesterday		
09_capstone_producer_operator_consumer	modified md	yesterday		
10_Art_Installation	Added more examples	yesterday		
11_BIG_SerpentinePavilion	Added more examples	yesterday		
12_MVRDV_Valley	Added some more markdowns to tutorials	yesterday		
_TEMPLATE	Add Intent to <algorithm> Python workshop: lessons 00-03	2 days ago		
README.md	modified md	yesterday		

3.3 IMPORT STATEMENTS: RHINO SCRIPT SYNTAX



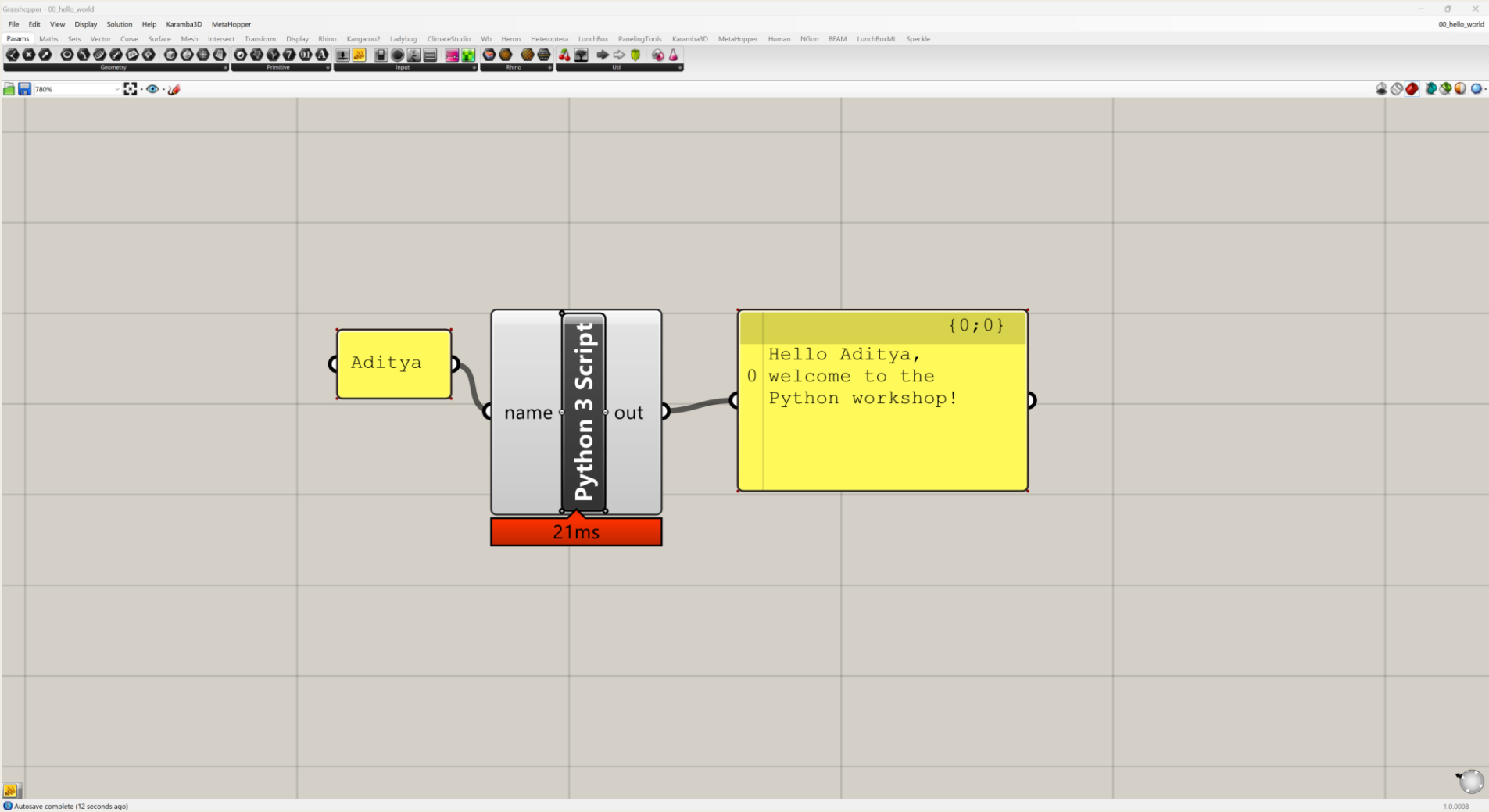
3.1 HELLO WORLD!

Github: Intent-to-Algorithm/RhinoScript editor/00_hello_world



3.3 RHINO SCRIPT COMPONENT

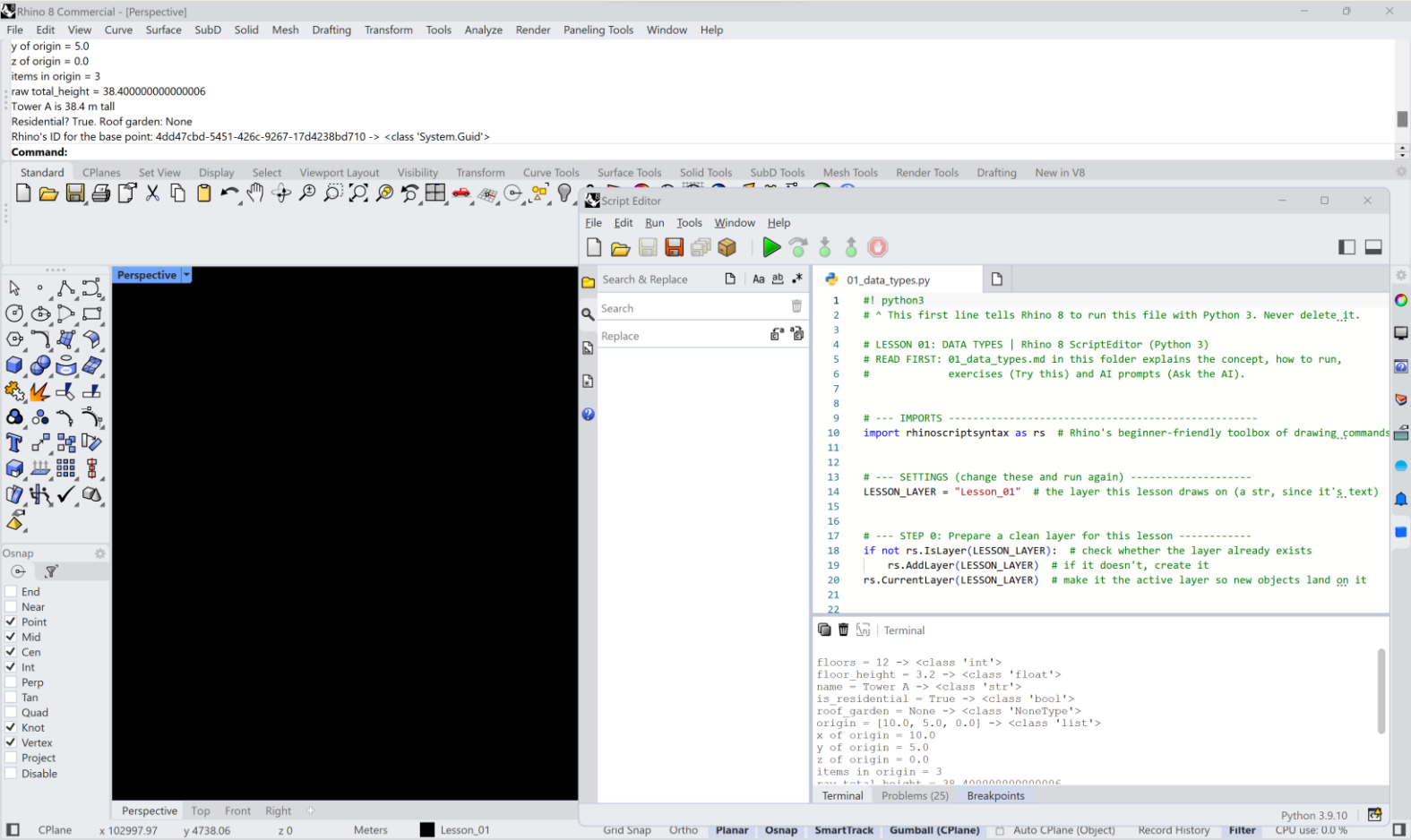
Grasshopper Component: Python3Script



Live Demo.

3.3 DATA TYPES IN PYTHON

Intent-to-Algorithm/RhinoScript editor/01_data_types



Data type	Description	Example
Integer (int)	A whole number	age = 25
Float (float)	A number with a decimal point	price = 9.99
Complex (complex)	A number with real and imaginary parts	z = 2 + 3j
String (str)	Text enclosed in quotes	name = "Ada"
Boolean (bool)	A true or false value	is_ready = True
List (list)	An ordered, changeable collection	scores = [10, 20, 30]
Tuple (tuple)	An ordered collection that cannot be changed	position = (4, 7)
Dictionary (dict)	A collection of key-value pairs	student = {"name": "Ada", "age": 25}
Set (set)	An unordered collection of unique items	colours = {"red", "blue"}
Range (range)	A sequence of integers	numbers = range(5)
Bytes (bytes)	An unchangeable sequence of bytes	data = b"Hello"
None (NoneType)	Represents the absence of a value	result = None

3.3 IMPORT STATEMENTS: RHINO SCRIPT SYNTAX

 **RhinoDeveloper**[®]

GuidesSamplesAPIVideosCommunity

RhinoScriptSyntax

Search

application
block
curve
dimension
document
geometry
grips
group
hatch
layer
light
line
linetype
material
mesh
object
plane
pointvector
selection
surface
toolbar
transformation
userdata
userinterface
utility
view

API References /

RhinoScriptSyntax

application

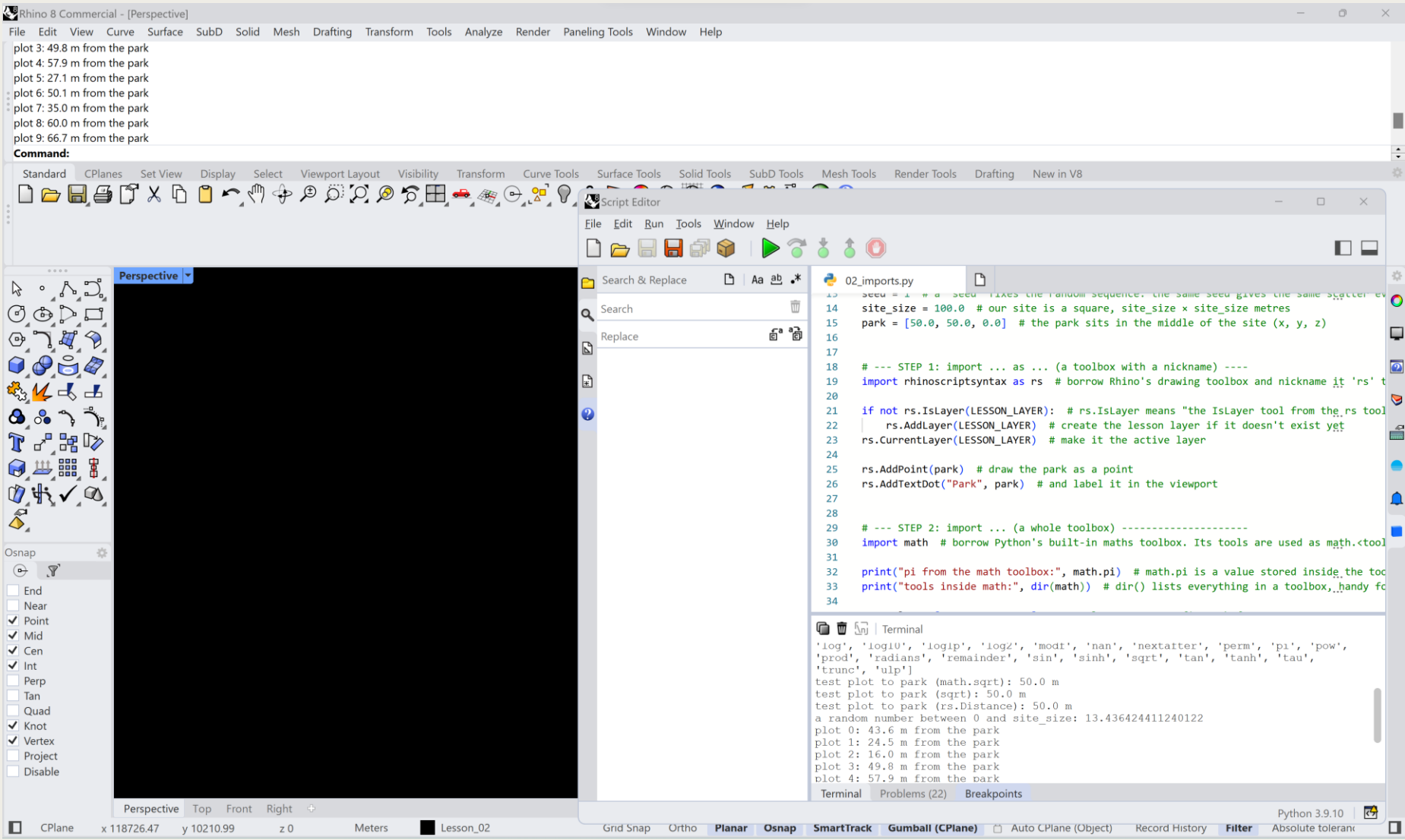
[AddAlias](#)
[AddSearchPath](#)
[AliasCount](#)
[AliasMacro](#)
[AliasNames](#)
[AppearanceColor](#)
[AutosaveFile](#)
[AutosaveInterval](#)
[BuildDate](#)
[ClearCommandHistory](#)
[Command](#)
[CommandHistory](#)
[DefaultRenderer](#)
[DeleteAlias](#)
[DeleteSearchPath](#)
[DisplayOleAlerts](#)
[EdgeAnalysisColor](#)
[EdgeAnalysisMode](#)
[EnableAutosave](#)
[EnablePlugIn](#)
[ExeFolder](#)
[ExePlatform](#)
[ExeServiceRelease](#)

<https://developer.rhino3d.com/api/RhinoScriptSyntax/>

Also Check Rhino Common

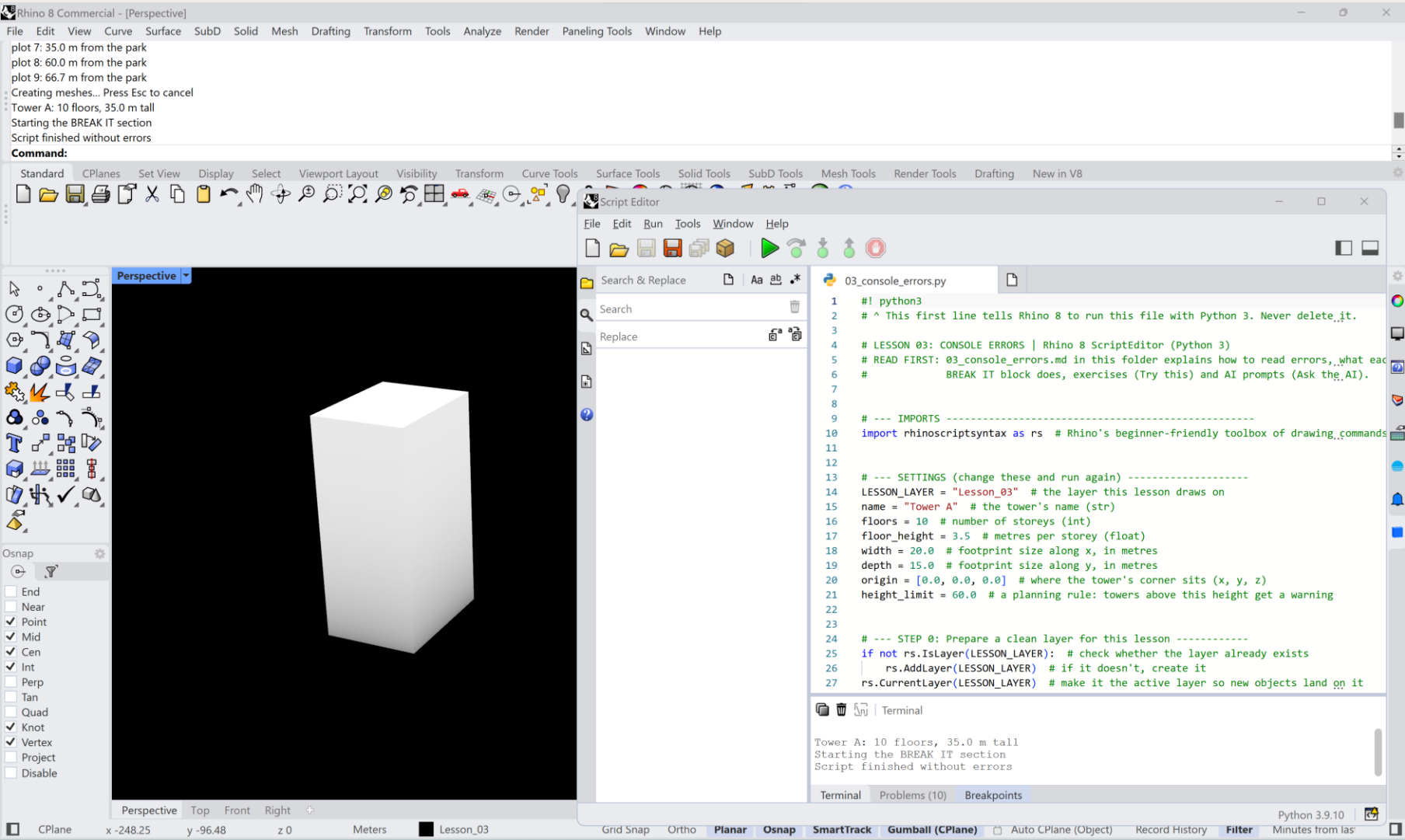
3.3 IMPORT STATEMENTS

Github: Intent-to-Algorithm/RhinoScript editor/02_imports



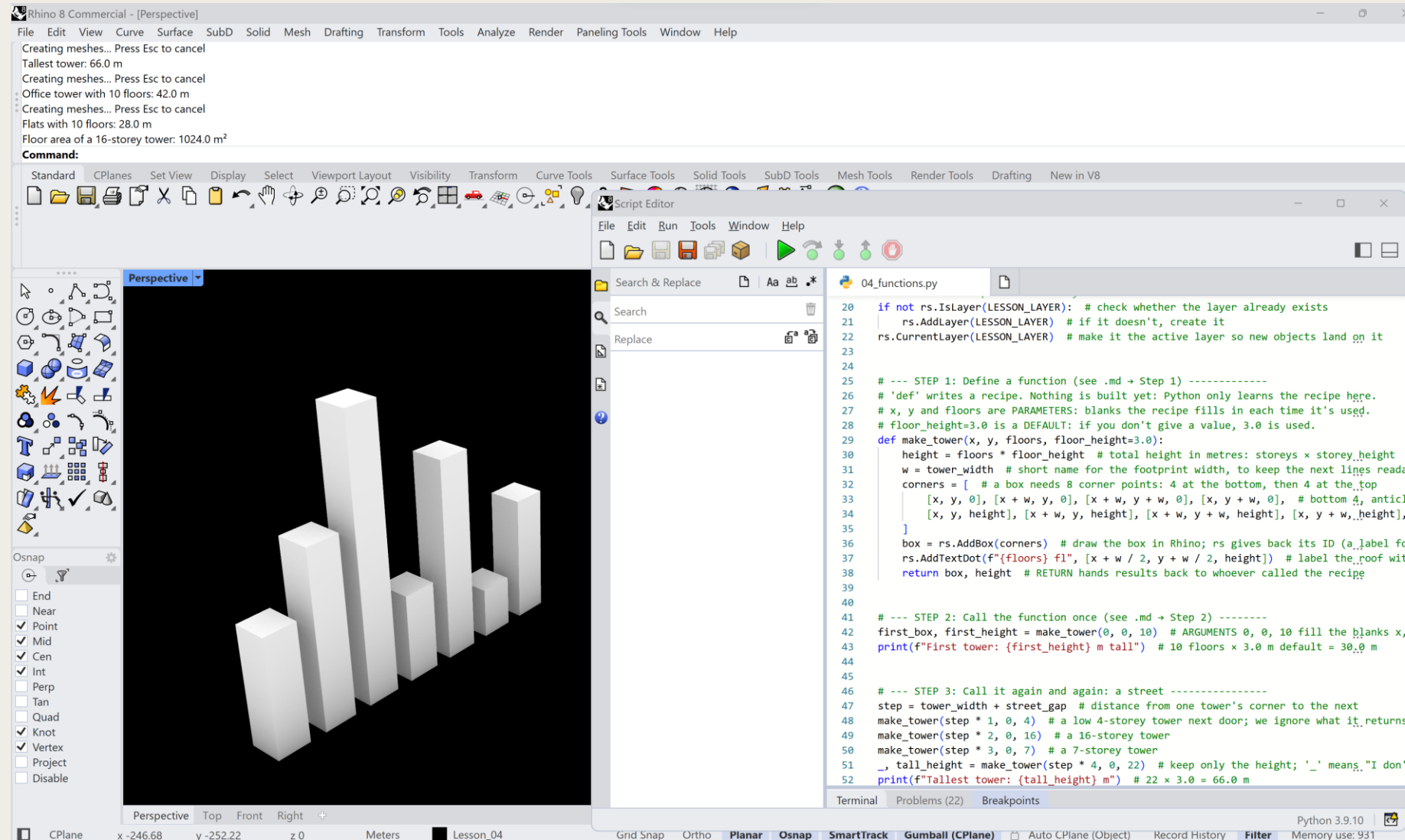
3.3 CONSOLE ERROR

Github: Intent-to-Algorithm/RhinoScript editor/03_console_errors



3.3 FUNCTIONS IN PYTHON

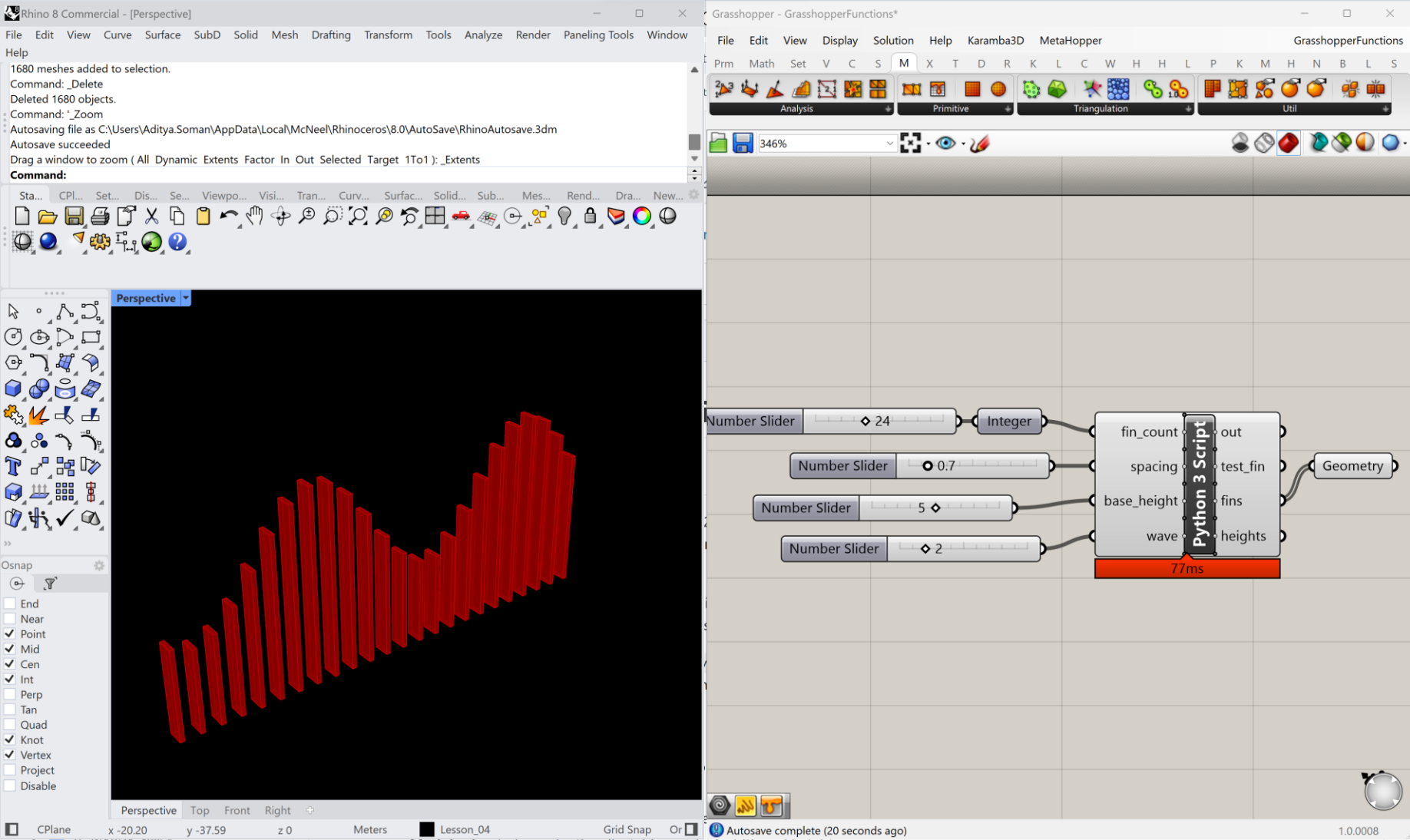
Github: Intent-to-Algorithm/RhinoScript editor/04_functions



A **function** in Python is a reusable block of code that performs a specific task. Use the keyword **def** to define it.

3.3 FUNCTIONS IN PYTHON

Github: Intent-to-Algorithm/Grasshopper scripts/04_functions



3.3 TYPICAL NATURE OF A GH SCRIPT

Import Statements

Indicating which external libraries,
we are using in Python



Functions

Steps of the process written down
as reusable functions



Outputs

Outputs designated to variables
that need to be copied on the
component

```
# --- IMPORTS -----  
import rhinoscriptsyntax as rs
```

```
def make_tower(x, y, floors, floor_height=3.0):  
    height = floors * floor_height # total height in metres: storeys x storey height  
    w = tower_width # short name for the footprint width, to keep the next lines readable  
    corners = [ # a box needs 8 corner points: 4 at the bottom, then 4 at the top  
        [x, y, 0], [x + w, y, 0], [x + w, y + w, 0], [x, y + w, 0], # bottom 4, anticlockwise  
        [x, y, height], [x + w, y, height], [x + w, y + w, height], [x, y + w, height], # top 4, same order  
    ]  
    box = rs.AddBox(corners) # draw the box in Rhino; rs gives back its ID (a label for the object)  
    rs.AddTextDot(f"{floors} fl", [x + w / 2, y + w / 2, height]) # label the roof with the floor count  
    return box, height # RETURN hands results back to whoever called the recipe
```

```
134  
135 meshes, colours, outlines, centres, tips, angles, heights = [], [], [], [], [], [], []  
136 info = ""  
137
```

3.3 SOME MORE EXAMPLES

Github: Intent-to-Algorithm/Grasshopper scripts/10_Art_Installation

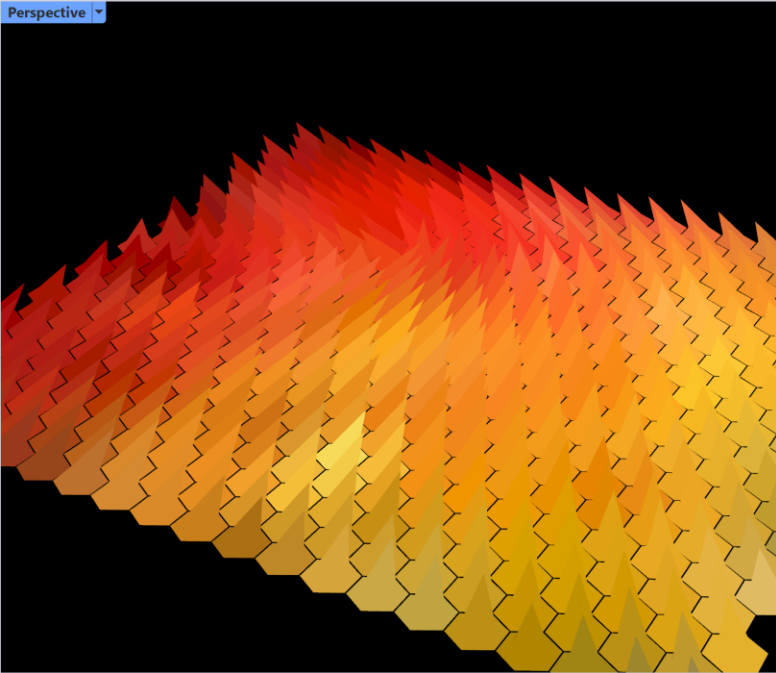
Rhino 8 Commercial - [Perspective]

File Edit View Curve Surface SubD Solid Mesh Drafting Transform Tools Analyze Render Paneling Tools Window Help

Office tower with 10 floors: 42.0 m
Creating meshes... Press Esc to cancel
Flats with 10 floors: 28.0 m
Floor area of a 16-storey tower: 1024.0 m²
7 polysurfaces added to selection.
Command: _Delete
Deleted 7 objects.
Command:

Sta... CPl... Set... Dis... Se... Viewpo... Visi... Tran... Curv... Surf... Solid... Sub... Mes... Rend... Dra... New...

Perspective



Perspective Top Front Right

Osnap

End

Near

Point

Mid

Cen

Int

Perp

Tan

Quad

Knot

Vertex

Project

Disable

CPlane

x -1689.57 y -897.32 z 0

Meters

Lesson_04

Grid Snap

Autosave complete (60 seconds ago)

1.0.0008

Grasshopper - art-installation*

File Edit View Display Solution Help Karamba3D MetaHopper

Prm Math Set V C S M X T D R K L C W H H L P K M H N B L S

Analysis Primitive Triangulation Util

250%

Number Slider 22

Number Slider 20

Number Slider 70

Number Slider 12

Number Slider 100

Number Slider 0

Number Slider 100

cols

rows

cell_size

height_min

height_max

rotation_deg

twist_deg

Python 3 Script

out

meshes

322ms

Panel

0 Mesh (V:54 F:18)

1 Mesh (V:54 F:18)

2 Mesh (V:54 F:18)

3 Mesh (V:54 F:18)

4 Mesh (V:54 F:18)

5 Mesh (V:54 F:18)

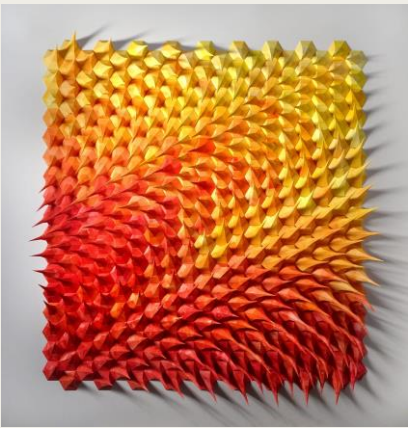
6 Mesh (V:54 F:18)

7 Mesh (V:54 F:18)

8 Mesh (V:54 F:18)

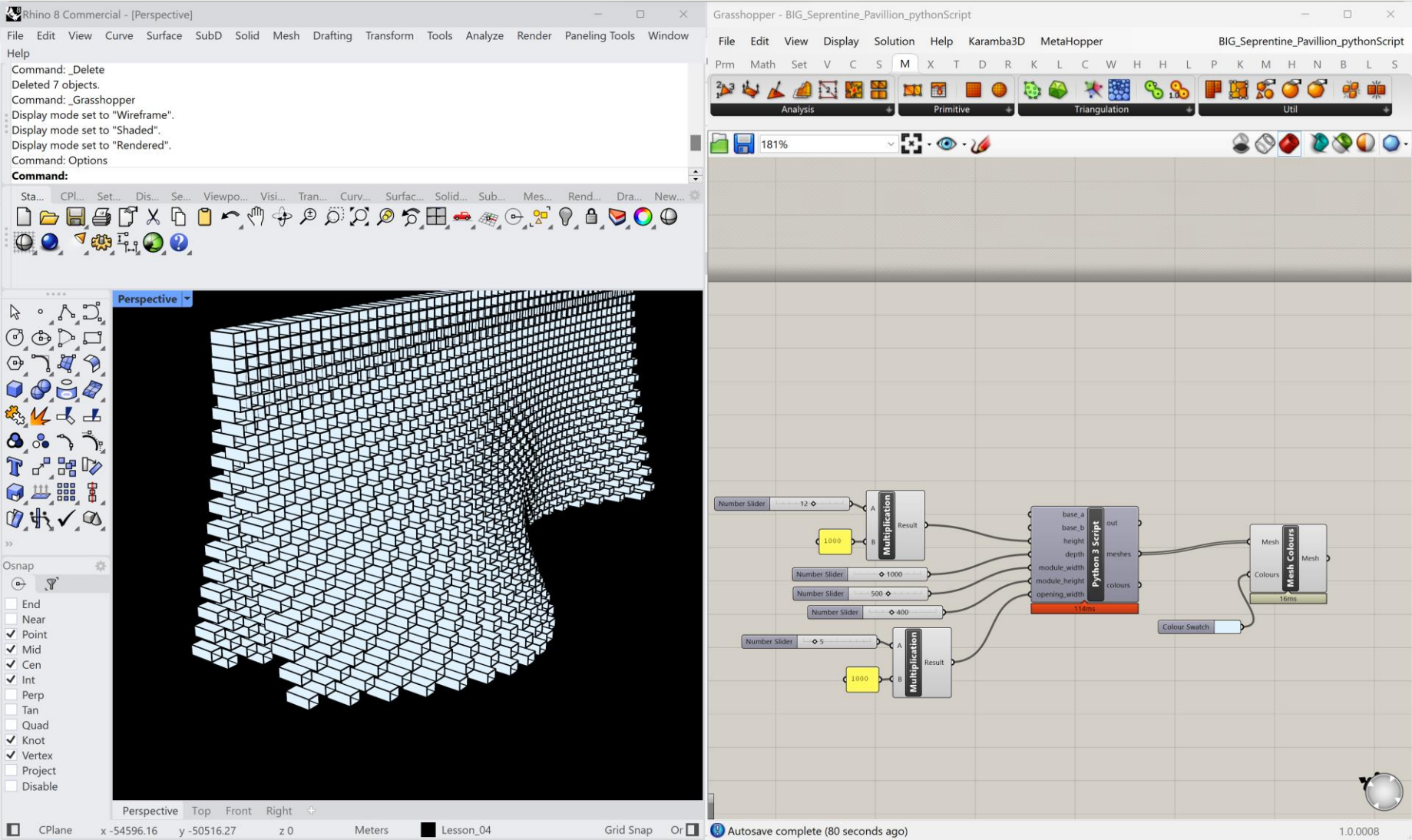
9 Mesh (V:54 F:18)

10 Mesh (V:54 F:18)



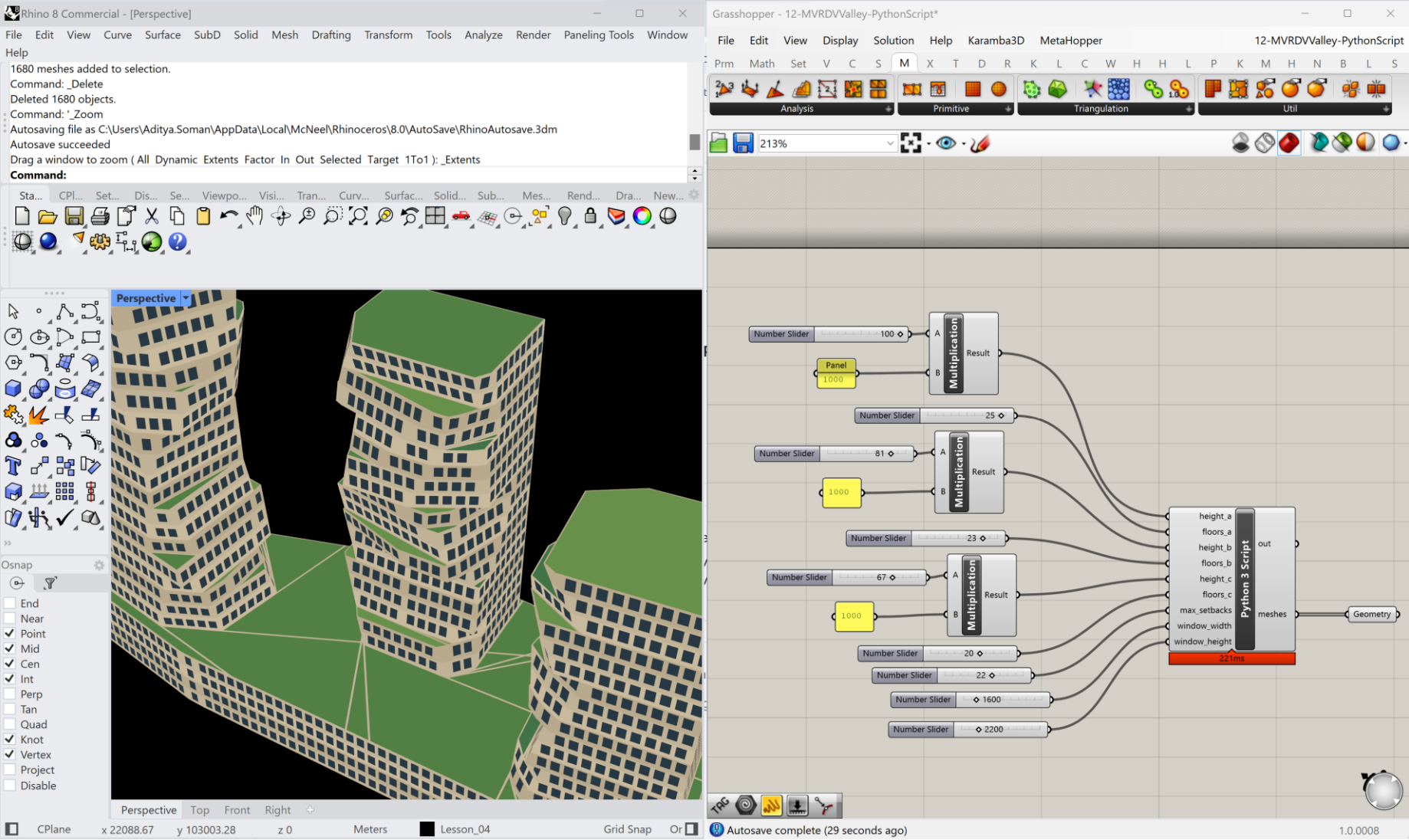
3.3 SOME MORE EXAMPLES

Github: Intent-to-Algorithm/Grasshopper scripts/11_BIG_SerpentinePavillion



3.3 SOME MORE EXAMPLES

Github: Intent-to-Algorithm/Grasshopper scripts/10_Art_Installation



RECAP

AFTER A SHORT BREAK...

04. AI ASSISTED SCRIPTING

3.4 TECHNIQUE 1 : {PROMPT}-SCRIPT-DEBUG

Github: Intent-to-Algorithm/AI-Assisted Scripting/Technique 1

Files

main

Go to file

Group-Assignments

Individual-Assignments

Intent-to-Algorithm

AI-Assisted Scripting/Techniqu...

Prompt_Script_Debug.md

Prompt_Script_Debug.py

Grasshopper scripts

Ideation-Exercises

Prompt guidelines

Python-Cheat Sheet

RhinoScript editor

data

.gitignore

README.md

UCL26Workshops-Intent-To-Algorithm / Intent-to-Algorithm / AI-Assisted Scripting / Technique 1 / Prompt_Script_Debug.md

Preview Code Blame 200 lines (143 loc) · 9.58 KB

Raw Copy Download Edit Menu

Consumer builds the rotated boxes

Step 1: Prompt

Copy this prompt into your AI agent (Claude, ChatGPT, Copilot, Gemini...) exactly as written.

Write a Python script for the Grasshopper "Python 3 Script" component in Rhino 8.

What the script should do:

- Create a grid of points on the XY plane.
- Place a box on each grid point.
- Use an attractor point to control each box:
boxes closer to the attractor are TALLER and MORE ROTATED,
boxes further away are shorter and less rotated.
- Rotate each box around its own centre, about the Z axis.

Inputs I want as component parameters:

- count_x (int): number of points in X
- count_y (int): number of points in Y
- spacing (float): distance between points
- box_size (float): width and depth of each box
- attractor (Point3d): the attractor point, which I will set in Rhino
- min_height, max_height (float): the height range of the boxes
- max_rotation (float): the largest rotation, in degrees

Outputs I want:

- points: the grid points
- boxes: the box geometry
- heights: the height of each box

Rules:

- Python 3 only (start the script with #! python3). Not IronPython or GhPython.
- Use only Rhino.Geometry and the Python standard library (math). No rhinoscriptsyntax, no numpy.
- Return Rhino.Geometry objects, not GUIDs.
- Use simple, readable code with a short comment on every line. I am a beginner.
- If the attractor is not connected, show a warning on the component instead of crashing.

Before the code, give me a table of the component inputs with:

Name	Type hint	Access (Item or List)	What to connect (slider range or Rhino point).
------	-----------	-----------------------	--

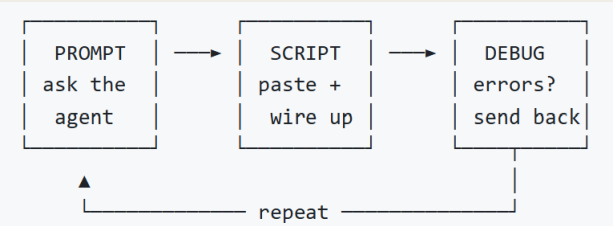
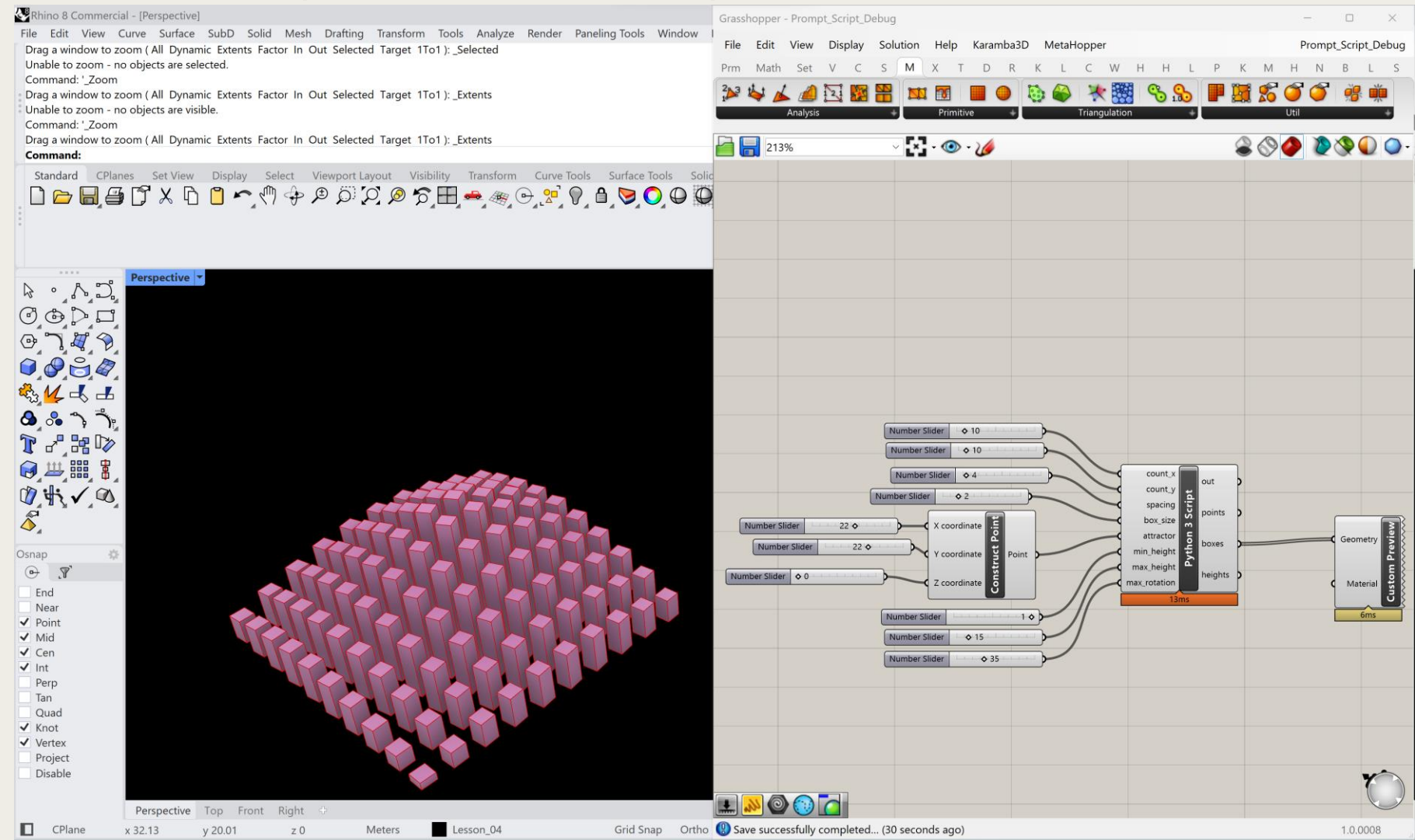
Then list the output names.

Tin: asking for the table first is the most important part. It tells you exactly how to set up the component in Step 2

Live Demo.

3.4 TECHNIQUE 1: {PROMPT}-SCRIPT-DEBUG

Github: Intent-to-Algorithm/AI-Assisted Scripting/Technique 1



Live Demo.

3.4 TECHNIQUE 2 : { PROMPT+ IMAGE/SKETCH} -SCRIPT-DEBUG

Github: Intent-to-Algorithm/AI-Assisted Scripting/Technique 2

Files

main

Go to file

Group-Assignments

Individual-Assignments

Intent-to-Algorithm

AI-Assisted Scripting

Technique 1

Technique 2

Prompt_Image_Script_Debug...

Prompt_Image_Script_Debug...

Prompt_Image_Script_Debug...

Sketch.png

Grasshopper scripts

Ideation-Exercises

Prompt guidelines

Python-Cheat Sheet

RhinoScript editor

data

.gitignore

README.md

UCL26Workshops-Intent-To-Algorithm / Intent-to-Algorithm / AI-Assisted Scripting / Technique 2 / Prompt_Image_Script_Debug.md

adityasoman Added new Task

dbc2e8f · 2 minutes ago History

Preview Code Blame 227 lines (164 loc) · 12.2 KB

Raw Copy Download Edit Menu

Technique 2: Prompt + Image → Script → Debug

In one line: give the AI agent a sketch as well as a prompt, ask for a Grasshopper Python script, then debug it, including comparing the result with the sketch.

The loop

```
graph LR; A[PROMPT + IMAGE sketch + text] --> B[SCRIPT paste + wire up]; B --> C[DEBUG errors? does it match the sketch?]; C -- repeat --> A;
```

What's new compared with Technique 1:

- A sketch shows the **shape and the rule** much faster than words. A sketch **doesn't** reliably show the **numbers**, so your text still has to give them.
- Debugging now has two kinds of problem: **errors** (the component turns red) and **mismatches** (it runs, but the result doesn't look like the sketch). For mismatches you send the agent a **screenshot** next to the sketch.

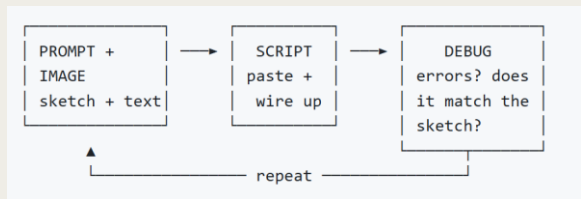
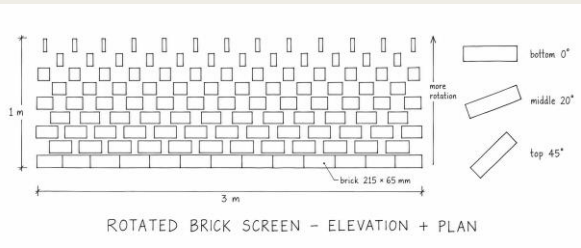
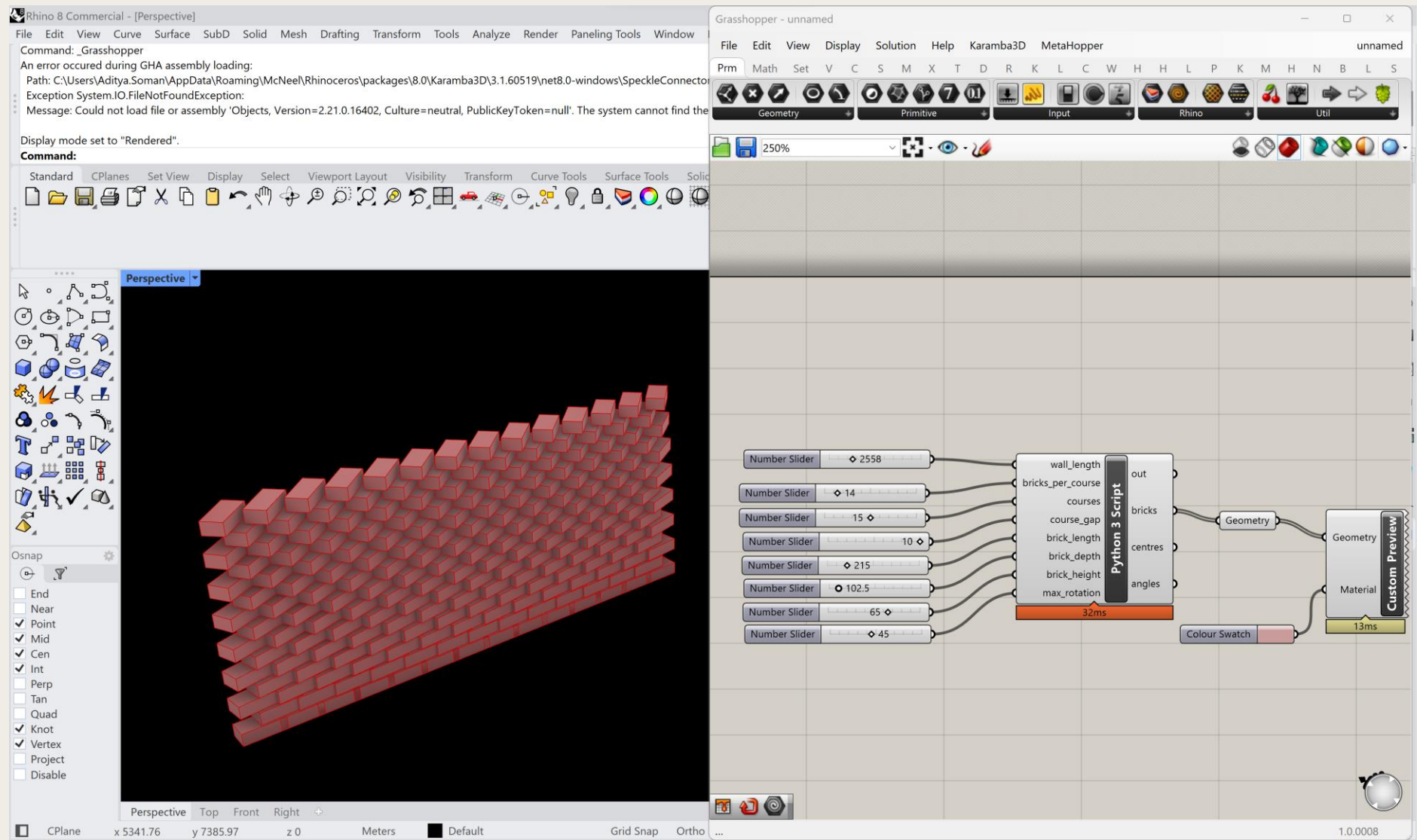
The task

Build a **rotated brick screen**: a wall of bricks in stretcher bond, where each course rotates its bricks a little more than the course below, opening gaps for light and air.

[Rotated brick screen sketch](#)

Step 1: Make the sketch

3.4 TECHNIQUE 2 : { PROMPT+ IMAGE/SKETCH} -SCRIPT-DEBUG



Live Demo.

3.4 TECHNIQUE 3 : { PROMPT+ SCRIPT + ANNOTATED RESULT} -SCRIPT-DEBUG

Files

main

Go to file

Group-Assignments

Individual-Assignments

Intent-to-Algorithm

AI-Assisted Scripting

Technique 1

Technique 2

Technique 3

Edited sketch.png

Prompt_Image_Script_Image...

Sketch_Over_Output.md

Sketch_Over_Output.py

Grasshopper scripts

00_hello_world

01_data_types

02_imports

03_console_errors

04_functions

05_conditionals

06_loops

07_classes

08_data_input_output

09_capstone_producer_operat...

10_Art_Installation

UCL26Workshops-Intent-To-Algorithm / Intent-to-Algorithm / AI-Assisted Scripting / Technique 3 / Sketch_Over_Output.md

adityasoman added new technique

Preview

Code

Blame

216 lines (152 loc) · 12.7 KB

Raw

Technique 3: Sketch Over Output → Modify → Debug

In one line: run your script, take a screenshot of the result in Rhino, draw your changes on top of it, and send the marked-up image to the AI agent with your current script so it can modify it.

The loop

RUN
current
script

→

SKETCH OVER
screenshot +
draw changes

→

PROMPT +
markup +
current code

→

DEBUG
does it
match?

↑ repeat

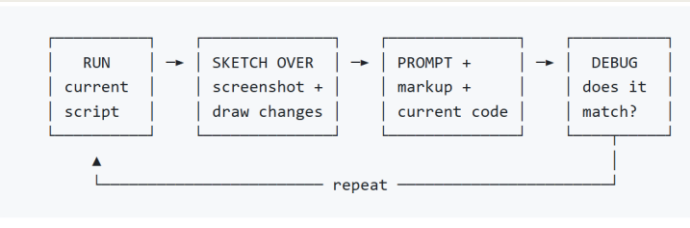
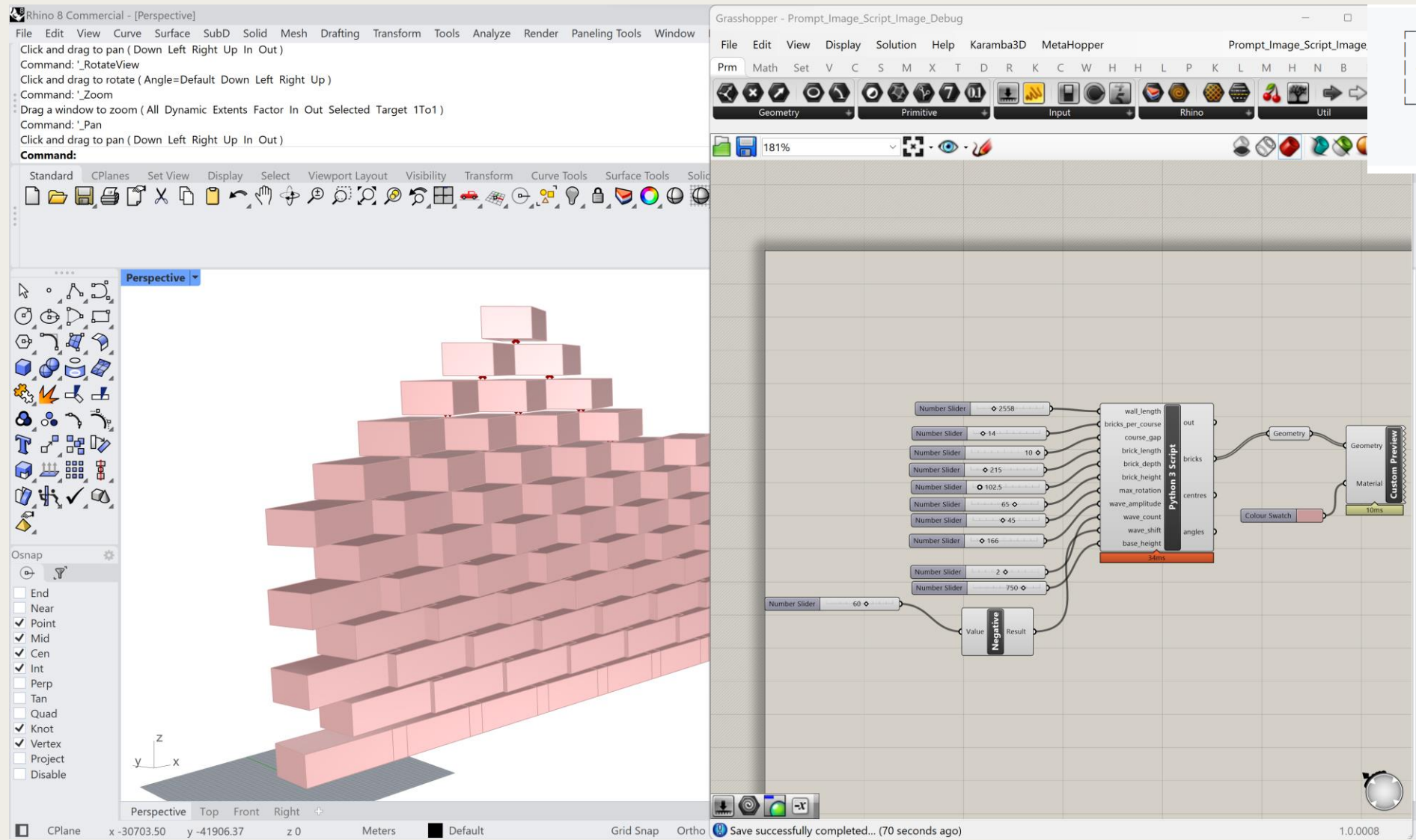
What's new compared with Techniques 1 and 2:

	Technique 1	Technique 2	Technique 3
You start from	nothing	nothing	a script that already works
You send	words	words + a sketch	words + your output with changes drawn on it + the current code
You ask for	a new script	a new script	a modified script

This is how designers usually work: you rarely start from a blank page. You look at what you have, draw over it ("this bit higher, remove this, curve this"), and iterate. The markup is a design crit on your own output.

Page 63

3.4 TECHNIQUE 3 : { PROMPT+ SCRIPT + ANNOTATED RESULT} -SCRIPT-DEBUG



EX03: USE TECHNIQUE ONE AND TRY TO WRITE AND RUN A SCRIPT THAT WORKS IN GRASSHOPPER

1. Craft a prompt of your choice or use the example one from git repo.
2. Ask AI agent to write code.
3. Paste the code inside Python component in GH.
4. If there are errors copy them and ask AI to debug
5. Continue this process till you have a working script

EX04: USE TECHNIQUE TWO AND TRY TO WRITE AND RUN A SCRIPT THAT WORKS IN GRASSHOPPER

1. Craft a prompt and add a reference Image/Sketch of your choice or use the example one from git repo.
2. Ask AI agent to write code.
3. Paste the code inside Python component in GH.
4. If there are errors copy them and ask AI to debug
5. Continue this process till you have a working script

EX05: USE TECHNIQUE THREE AND TRY TO WRITE AND RUN A SCRIPT THAT WORKS IN GRASSHOPPER

1. Use your Script from EX04 and sketch some modifications on output screenshot and send script, sketch and prompt to modify.
2. Ask AI agent to do the modifications.
3. Paste the code inside Python component in GH.
4. If there are errors copy them and ask AI to debug
5. Continue this process till you have a working script

RECAP

AFTER A SHORT BREAK...

04. GITHUB DEMO

Files

main

Go to file

Group-Assignments

Individual-Assignments

Intent-to-Algorithm

Grasshopper scripts

Ideation-Exercises/Exercise-01

Prompt guidelines

RhinoScript editor

data

.gitignore

README.md

UCL26Workshops-Intent-To-Algorithm / Individual-Assignments /

↑ Top

README.md

modified md

11 hours ago

README.md

Individual assignments

Upload your **own** work here. Each student has one folder, and inside it there is one subfolder per assignment.
Group work goes in `../Group-Assignments/`, not here.

Folder structure

Individual-Assignments/

└─ Firstname-Lastname/ ← one folder per student, named after you

├─ Assignment-01/

├─ Assignment-01.md ← required: describes your work (template below)

├─ my_script.py ← your code

└─ screenshot.png ← images of the result

└─ Assignment-02/

├─ Assignment-02.md

└─ ...

Rules

1. Create one folder named after yourself: `Firstname-Lastname`, e.g. `Jane-Smith`. Use hyphens, not spaces, and use the same name every time.

2. Create one subfolder per assignment: `Assignment-01`, `Assignment-02`, ... Always use two-digit numbers.

3. Every assignment subfolder needs a Markdown file with the same name as the folder, e.g. `Assignment-01/Assignment-01.md`. Use the

EXERCISE 06: INSIDE INDIVIDUAL-ASSIGNMENTS FOLDER IN GITHUB CREATE A FOLDER OF USING THE NAMING CONVENTION AND ADD A .MD FILE

1. CLONE GITHUB REPO
2. CREATE FOLDER WITH RIGHT NAME
3. CREATE THE .MD FILE WITH THE RIGHT NAME
4. CREATE YOUR BRANCH
5. PUSH THE CHANGES TO THE BRANCH
6. MAKE A PULL REQUEST

NEXT UP ...

INDIVIDUAL ASSIGNMENT